



POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Zant: Lightweight Neural Network Inference Engine for Micro-controllers

TESI DI LAUREA MAGISTRALE IN
COMPUTER ENGINEERING - INGEGNERIA INFORMATICA

Author: **Mirko Calvi**

Student ID: 10724100
Advisor: Prof. Salice Fabio
Co-advisors: Massimo Pavan
Academic Year: 2024-25

Abstract

Deploying deep neural networks on microcontrollers (MCUs) requires a fundamental shift from general-purpose AI frameworks toward embedded-specific priorities such as static memory bounds, deterministic execution, and energy-efficient quantization. Existing solutions—including interpreter-based runtimes, compiler stacks, and vendor libraries—often trade off one or more of these properties, introducing runtime overheads, portability constraints, or tooling complexity.

This thesis presents Zant, a Zig-based, ONNX-native compiler framework for MCU-scale inference that produces fully static, memory-safe, and verifiable inference libraries. Zant compiles ONNX models into deterministic code with compile-time graph scheduling, buffer reuse via tensor lifetime analysis, and memory sectioning for execute-in-place (XIP) deployments. It supports both Quantize-Dequantize (QDQ) and operator-oriented quantization (OQ) schemes, enabling hybrid precision strategies tuned to MCU constraints.

Zant’s pipeline performs model simplification, pattern-based fusion, activation scheduling, and flash-aware section placement entirely at compile time. The resulting artifacts—small, linkable libraries with no runtime dependencies—integrate seamlessly into embedded firmware and continuous integration (CI) workflows. Experimental results across Cortex-M and x86 targets demonstrate how Zant positions itself with respect to existing frameworks while offering superior predictability and transparency.

This work positions Zant as a modular, portable foundation for deterministic inference on deeply embedded platforms, bridging the gap between modern ML tooling and the hard constraints of edge deployments.

Keywords: TinyML, microcontrollers, neural network inference, ONNX, quantization, edge computing

Abstract in lingua italiana

L'esecuzione di reti neurali profonde su microcontrollori (MCU) richiede un cambiamento radicale rispetto ai tradizionali framework di intelligenza artificiale general-purpose, in favore di priorità specifiche per i sistemi embedded, come limiti statici di memoria, esecuzione deterministica e quantizzazione. Le soluzioni esistenti, tra cui runtime basati su interprete, compilatori e librerie fornite dai vendor, spesso sacrificano uno o più di questi requisiti, introducendo overhead a runtime, vincoli di portabilità o una complessità eccessiva degli strumenti.

Questa tesi presenta Zant, un compilatore open-source basato su Zig e nativamente compatibile con ONNX, progettato per l'inferenza su MCU. Zant genera librerie statiche, sicure in memoria e verificabili, compilando modelli ONNX in codice deterministico tramite pianificazione del grafo a tempo di compilazione, riutilizzo dei buffer tramite analisi del ciclo di vita dei tensori, e suddivisione della memoria per il supporto all'esecuzione diretta da flash (XIP).

La pipeline di Zant esegue semplificazione del modello, fusioni basate su pattern, pianificazione delle attivazioni e posizionamento delle sezioni in flash a tempo di compilazione. Gli artefatti risultanti—librerie leggere, collegabili e prive di dipendenze a runtime—si integrano facilmente nei firmware embedded e nei workflow di continuous integration (CI). I risultati sperimentali su target Cortex-M e x86 dimostrano come Zant si rapporta a framework esistenti, offrendo al contempo maggiore prevedibilità e trasparenza.

Questo lavoro propone Zant come una base modulare e portabile per l'inferenza deterministica su piattaforme embedded, colmando il divario tra gli strumenti di machine learning moderni e i vincoli stringenti dei sistemi edge.

Parole chiave: TinyML, microcontrollori, reti neurali, ONNX, quantizzazione, edge computing

Contents

Abstract	i
Abstract in lingua italiana	iii
Contents	v
Introduction	1
1 Related Works	7
1.1 Interpreter-Based Inference on Microcontrollers	7
1.2 Compiler-Based Stacks and Autotuning	8
1.3 Vendor Toolchains and Kernel Libraries	8
1.4 IR-to-C Code Generation Pipelines	9
1.5 System–Model Co-Design for TinyML	9
1.6 Distributed/Collaborative Inference as Context	10
1.7 Where Existing Solutions Fall Short for MCU-Scale Deployment	10
1.8 Zant’s Positioning and Advantages	11
1.9 Quantization as a First-Class Optimization Axis	12
1.10 ONNX as the Interoperability Backbone	13
1.11 Summary	15
2 Methodology	17
2.1 Zant Workflow	17
2.1.1 Model pre-processing	19
2.1.2 IR Generator	20
2.1.3 IR Optimizer	22
2.1.4 Graph Linearization	22
2.1.5 Memory Planning Strategies	25
2.1.6 Code Generator	27

2.1.7	Correctness Validation Against ONNX Runtime	28
2.1.8	Compiling for a Target Architecture	28
2.2	Compile-Time Graphs Navigation for Deterministic Firmware	29
2.3	Zig as the compiler of choice	30
3	Experimental Results	33
3.1	Zant Static vs Dynamic Memory Allocation: Inference Time on x86_64 . .	36
3.2	Zant Static vs Dynamic Memory Usage: RAM x86_64	37
3.3	Zant vs TensorFlow Lite Micro (TFLM) on ARM Cortex-M7	38
3.4	Discussion of Experimental Results	45
3.4.1	Static vs Dynamic Allocation in Zant	45
3.4.2	Zant vs TensorFlow Lite Micro	45
3.4.3	Scalability Across Model Families	46
4	Conclusions and future developments	47
4.1	Summary of Contributions	47
4.2	Hardware Awareness Without Lock-In	48
4.3	Distributed and Collaborative Inference: Context for MCUs	49
4.4	Limitations	49
4.5	Future Developments	50
4.5.1	Kernel Optimization and Backend Integration	50
4.5.2	Richer Graph Transformations and Scheduling	50
4.5.3	Dynamic-Shape Support Under Bounded Resources	50
4.5.4	Energy-Aware Evaluation and Co-Design	51
4.5.5	Tooling, Diagnostics, and Developer Experience	51
4.6	Closing Remarks	51
	Bibliography	53
	List of Figures	57
	List of Tables	59
	Acknowledgements	61

Introduction

The deployment of deep neural networks on resource-constrained microcontrollers (MCUs) has evolved from an aspiration into a mature engineering discipline. This field is not only a miniaturized version of mobile or server AI; it follows a distinct optimization landscape where *static memory planning*, *deterministic execution*, and *energy-efficient arithmetic* outweigh throughput-oriented concerns typical of data-center deployments. Consequently, toolchains and runtimes in general-purpose environments often underperform or fail in TinyML.

The main reason why this happens is due to a fundamental mismatch between the assumptions these toolchains make and the constraints imposed by MCU-class devices. General-purpose deep learning frameworks prioritize flexibility, dynamic memory allocation, and runtime shape inference to accommodate a wide range of models and use cases. In contrast, MCUs demand compile-time determinism, fixed memory footprints, and minimal control-flow overhead. Features like dynamic tensor resizing, on-the-fly operator fusion, or just-in-time compilation—which are common in server or mobile AI—are impractical or even impossible in microcontroller environments.

This mismatch results in bloated binaries, inefficient memory usage, and unpredictable latency when general-purpose tools are forced into the embedded domain. Even specialized converters or deployment pipelines (such as TensorFlow Lite Micro or CMSIS-NN) [7] [15] often require non-trivial adaptation, model rewriting, or handcrafted optimizations to meet strict deployment targets in terms of SRAM, flash size, or inference time.

Motivations for a New Path in Edge Inference

Edge intelligence at MCU scale is driven by three converging forces. First, demand is rising: IoT sensors perform predictive maintenance with bounded latency; wearables and medical devices run always-on inference for privacy [14, 18]; and robotics and home automation demand deterministic control loops. Second, transmitting data to the cloud is increasingly costly in energy and bandwidth, while privacy regulations favor local processing [18, 28]. Third, the MCU hardware base—Cortex-M and small RISC-V cores—offers

limited SRAM (64–512 kiB) and flash (hundreds of kilobytes to a few megabytes), often lacking floating-point units, caches, or MMUs [5, 11]. These constraints redefine how inference must be deployed.

What distinguishes TinyML from general-purpose machine learning is its focus on compile-time determinism and energy efficiency. MLPerf Tiny codifies representative MCU workloads—such as keyword spotting and visual wake words—using end-to-end metrics designed to capture realistic deployment conditions [5]. In order to enable the execution of these tasks, techniques such as weight quantization and 8-bit operators have become the most used strategy. Hardware studies consistently show that integer arithmetic is vastly cheaper than floating point; in Horowitz’s analysis, 8-bit operations can be up to two orders of magnitude more energy-efficient than FP32 equivalents [11, 29]. Hence, *precision scaling* serves as both a compression method and a power-management strategy.

Three bottlenecks make MCU inference hard to achieve. (1) Activation memory often exceeds SRAM capacity even when weights fit in flash; peak activations (i.e., the largest set of intermediate values in memory during inference) determine feasibility. (2) Limited bus bandwidth and small caches make layout (how tensors are arranged in memory) and reuse (how often intermediate results can be used without recomputation) critical for latency and energy. (3) Dynamic runtimes introduce jitter (variable timing from run to run) and overhead, which complicate real-time scheduling.

Existing frameworks reveal the trade-offs involved. Interpreter-based systems like TensorFlow Lite Micro (TFLM) [7] and ONNX-Runtime [21] favor portability and ease of iteration but incur runtime overhead from runtime graph navigation.

Compiler-based systems such as TVM [6] achieve high performance via automated code generation and autotuning, but their complexity can be a barrier in TinyML projects because they often require transparent behavior, minimal dependencies, and fully predictable execution, which are hard to ensure when relying on opaque code generation pipelines or large host toolchains. [6].

Vendor kernel libraries such as CMSIS-NN [15] provide highly efficient implementations of individual neural operations—referred to as *kernels*, but delegate critical aspects such as graph-level scheduling, memory planning, and portability to the developer [15]. Graph scheduling governs the execution order of layers within the model, while memory planning involves assigning buffers to tensors in a manner that avoids overlap and optimizes limited resources. Portability ensures that a model can be deployed across diverse hardware platforms without requiring substantial modification. Collectively, these tasks constitute the broader challenge of *whole-network integration*, which is frequently left to be addressed

manually, thereby increasing the development burden and potential for error.

Zant: Deterministic Deployment with ONNX Semantics and Hardware Awareness

In this thesis we present Zant, a novel Zig-based open-source inference engine and code-generation framework, that adopts a compiler-oriented approach for ONNX graphs [33]. It parses ONNX models, converts them into a deployment-oriented intermediate representation with explicit tensor metadata, and generates target-specific code. Preserving ONNX semantics ensures compatibility with existing quantization and calibration tools such as Microsoft Olive [20]. By default, Zant adopts Quantize-Dequantize (QDQ) but can selectively use operator-level quantization (OQ) for optimized paths.

Zant improves on existing approaches by:

- Performing all major optimizations at compile time: layout, memory allocation, and operator scheduling are resolved statically.
- Implementing tensor lifetime analysis, allowing safe buffer reuse and fixed SRAM footprints.
- Supporting execute-in-place (XIP) from flash to reduce SRAM usage.
- Replacing interpreters with statically compiled Zig code and direct function calls, eliminating dynamic dispatch.
- Remaining interoperable with ONNX and its tooling ecosystem (e.g., quantization, calibration).
- Supporting both QDQ and QOperator quantization schemes depending on deployment needs.
- Enabling hardware-specific backends like CMSIS-NN without locking users into any one vendor.
- Providing verifiable outputs: deterministic static libraries that integrate into C/C++ firmware and CI pipelines.

Zant performs all major optimizations at compile time, ensuring deterministic and memory-efficient inference. Complete knowledge of the computation graph enables *lifetime analysis* of tensors, allowing buffer reuse and guaranteeing a fixed SRAM footprint before deployment. The compiler fuses operators, allocates buffers statically, and emits efficient

Zig code that replaces dynamic interpretation with direct function calls. Through *memory sectioning* and *execute-in-place (XIP)*, weights can be accessed directly from flash, improving latency and freeing SRAM for activations.

Zant’s hardware awareness is modular and vendor-neutral: on ARM Cortex-M it can exploit DSP extensions and CMSIS-NN kernels, while on RISC-V and heterogeneous SoCs it can support equivalent SIMD or accelerator libraries through compile-time configuration. This design balances portability and performance, positioning Zant between interpreter-based frameworks like TFLM and compiler-based systems such as TVM.

To strengthen developer confidence, Zant emphasizes verifiability and maintainability. *ONNX-consistent testing* prevents semantic drift, individual operators or subgraphs can be compiled in isolation, and deterministic static libraries integrate easily with embedded build systems and continuous-integration pipelines. Relative to existing approaches, Zant combines the predictability of static compilation with selective use of vendor kernels, targeting inference on MCUs with limited memory and no operating system. Its core priorities—determinism and minimal dependencies—make it well-suited for safety-critical and resource-constrained environments.

5 Contributions

The contributions of this thesis are fourfold:

First, we present Zant, a compile-time, ONNX-native deployment framework that generates static, linkable libraries with deterministic memory footprints for MCU targets [33].

Third, we formalize a *section-aware memory placement* method for XIP-based deployments, placing parameters in flash-backed sections and reserving SRAM for activation pools under compile-time lifetime bounds [19, 24].

Fourth, we integrate *pattern-based fusion* and *lifetime reuse* into the optimization stage of the Zant pipeline, aligning algebraic transforms with buffer planning and producing reproducible, auditable artifacts suited for embedded CI and safety-critical workflows [17, 23].

Structure of the Thesis

In *Related Works* chapter 1 situates Zant among interpreter-based runtimes, compilers, and vendor stacks [6, 7, 15, 17]. It follows *Methodology* chapter 2 focus on explaining how Zant operates as a multi-stage compilation pipeline, transforming a hardware-agnostic

ONNX graph into a target-specific static library. The *Experimental Results* chapter 3 benchmarks latency, energy, and memory under varying quantization granularities and fusion settings across Cortex-M, RISC-V, and x86 harnesses. The *Conclusion* 4 discusses limitations and future work, including automated tiling/streaming, broader accelerator backends, and formal SRAM upper bounds for dynamic-shape variants.

1 | Related Works

Edge inference on microcontrollers occupies a distinctive niche at the intersection of systems, compilers, and machine learning. This chapter positions *Zant* within that landscape by reviewing representative approaches across five families: (i) interpreter-based inference runtimes for embedded systems, (ii) compiler-based stacks and autotuning frameworks, (iii) vendor toolchains and kernel libraries, (iv) code-generation pipelines and IR-to-C compilers, and (v) system-model co-design for TinyML. We analyze their strengths and weaknesses with respect to the constraints that dominate microcontroller deployments—namely static memory bounds, deterministic execution, and energy-aware data movement—and articulate how *Zant*’s design choices answer the shortcomings observed in prior work.

1.1. Interpreter-Based Inference on Microcontrollers

Interpreter-based frameworks provide a uniform runtime that loads a model representation and dispatches operators to platform-specific kernels. *TensorFlow Lite Micro* (TFLM) is the canonical example in the TinyML context, designed expressly for highly constrained devices [7]. Its contributions include a compact interpreter, a curated kernel library, and portability across a wide range of microcontrollers. Empirical studies highlight low runtime overheads for small networks and a development experience that eases cross-platform experimentation [7].

Despite these strengths, interpreter-based systems face structural limitations in MCU settings. First, the presence of a graph interpreter, dispatcher tables, and operator metadata introduces a persistent *control-plane overhead* that scales with model complexity. Even carefully engineered allocators and arena planning cannot eliminate dynamic behaviors entirely; in practice, the combination of runtime scaffolding, dynamic dispatch, and general-purpose memory allocators [7] can inflate static memory budgets. Dynamic dispatch refers to invoking functions indirectly (e.g., through function pointers or virtual tables), which increases flexibility but also adds call overhead and code size. General allocators, on the other hand, rely on heap-based memory management that is inherently

nondeterministic in both size and timing. Second, achieving deterministic execution time under mixed workloads (sensing, control, inference) is complicated by residual dynamic dispatch and the need to maintain interpreter state. Third, although TFLM supports quantization, the integration of advanced ONNX-centric flows (e.g., QDQ graphs with blocked or per-axis schemes) requires additional tooling outside the core interpreter. In short, the very flexibility that makes interpreter-based systems attractive for prototyping can be a liability when SRAM budgets are tight and real-time guarantees are paramount.

1.2. Compiler-Based Stacks and Autotuning

Compiler-based approaches treat the model as a program to be optimized and compiled for the target. *TVM* exemplifies this paradigm, applying schedule transformations, tensorization, and autotuning to produce high-performance kernels across diverse hardware [6]. *TVM* demonstrates that automated compilation can often match or surpass hand-optimized code, a result amplified by learned cost models and increasingly expressive scheduling languages.

However, critical challenges persist when porting the *TVM* philosophy directly to microcontroller deployments. The autotuning process presupposes time, compute, and power resources rarely available in embedded development, especially for safety-critical or field devices. Debugging and validating schedule decisions demand specialized expertise, and the stack’s breadth can be heavy for teams whose primary goal is to ship deterministic firmware artifacts with tightly bounded SRAM and flash. While *TVM* continues to make inroads into edge-device targets, a microcontroller-focused deployment often benefits from a narrower scope: fewer degrees of freedom, more compile-time decisions fixed by design, and artifacts that integrate seamlessly into C/C++ build systems with static memory guarantees.

1.3. Vendor Toolchains and Kernel Libraries

Whereas compilers optimize across operators and memory schedules, vendor toolchains and kernel libraries focus on efficient *per-operator* implementations and tight integration with device features. *CMSIS-NN* for ARM Cortex-M provides hand-tuned integer kernels that routinely improve both latency and energy efficiency relative to generic implementations [15]. Similarly, microcontroller vendors provide proprietary code-generation suites (e.g., *STM32Cube.AI*) that map trained models to C code, often with platform-specific optimizations and accelerator offload.

These tools are invaluable, but they are not full deployment stacks. Kernel libraries stop at the operator boundary: it is the developer’s responsibility to schedule graphs, plan memory lifetimes, and reconcile tensor layouts across kernels. Vendor code generators can bridge that gap for single-vendor ecosystems, yet they typically trade transparency and portability for convenience. When a product line includes multiple MCU families—or when portability to RISC-V and x86 host harnesses is required for CI and testing—platform-specific tools become friction points. In all cases, the scarcity of explicit, auditable *memory sectioning* (e.g., `.rodata/.qspidata` for execute-in-place) in end-user outputs can complicate precise SRAM/flash budgeting at link time.

1.4. IR-to-C Code Generation Pipelines

Code-generation pipelines that translate a model IR directly into C or C-like code target many of the same goals as Zant: small artifacts, no dynamic runtime, and tight integration with firmware. The *onnx2c* project is emblematic, converting ONNX graphs into portable C functions intended for inclusion in MCU projects [13]. These systems generally eliminate interpreter overhead and enable deep inspection by firmware engineers.

Their limitations emerge around *graph-level* optimization and *quantization depth*. Many IR-to-C projects prioritize operator coverage and portability over aggressive pattern fusion, static lifetime analysis, and hybrid quantization flows per ONNX QDQ/QOperator semantics. Memory sectioning for XIP and linker-directed placement (e.g., routing large parameters to QSPI-backed regions while reserving SRAM pools for activations) is also a frequent gap. As a result, raw code generation without the surrounding compile-time planning often leaves performance and memory savings unrealized.

1.5. System–Model Co-Design for TinyML

System–model co-design frameworks jointly optimize neural architectures and inference engines under microcontroller constraints. *MCUNet* (TinyNAS + TinyEngine) is a leading example, achieving ImageNet-scale inference on off-the-shelf MCUs by pairing NAS-driven architectures with a lightweight engine that schedules memory reuse and minimizes activation peaks [17]. The literature underscores two lessons crucial for MCU deployments: (i) *activation memory* is often the dominant bottleneck; (ii) careful *graph scheduling and fusion* can convert infeasible models into feasible ones without altering accuracy. The MCUNet co-design of network and inference engine is shown in Fig. 1.1 [17].

Co-design is powerful but brings its own trade-offs. Integrating NAS into conventional

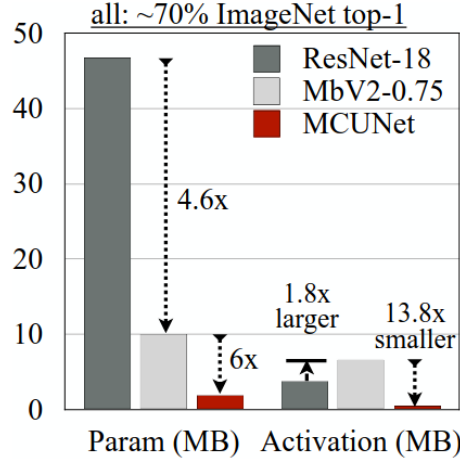


Figure 1.1: MCUNet co-design: TinyNAS (architecture search) + TinyEngine (memory-adaptive inference). Demonstrates joint hardware–software optimization for MCU deployment [17].

product cycles may be impractical, and the approach can entangle the deployment stack with specific model families. Moreover, while TinyEngine demonstrates compelling scheduling and memory tactics, portability across instruction sets and vendor ecosystems requires additional engineering effort when moving beyond the original ARM targets.

1.6. Distributed/Collaborative Inference as Context

A parallel thread of research explores *split computing* and *early exiting*: executing easy cases on-device with small subgraphs while offloading hard inputs to edge servers or the cloud [14, 18]. These methods yield substantial latency and energy gains under variable networks and can extend the capabilities of MCU-class devices by sharing the inference burden. Although not a replacement for efficient on-device execution, they provide architectural context: a robust deployment stack must produce a compact, deterministic local component that can compose with larger distributed systems.

1.7. Where Existing Solutions Fall Short for MCU-Scale Deployment

Across the families above, four structural limitations recur:

1. **Residual runtime overhead and nondeterminism.** Interpreter-based designs retain dispatch tables and metadata that scale with model complexity, complicating

hard SRAM bounds and creating latency jitter [7].

2. **Tooling complexity versus embedded ergonomics.** Autotuning compilers deliver peak performance but demand expertise and resource budgets misaligned with many embedded workflows; debugging schedule-induced regressions can be costly [6].
3. **Portability lock-in.** Vendor generators excel within an ecosystem but impede cross-architecture portability and auditability; kernel libraries alone do not solve graph scheduling and memory lifetime planning [15].
4. **Missing memory sectioning and lifetime proofs.** Many pipelines lack explicit *compile-time* memory plans with section-aware placement (e.g., `.rodata/.qspidata` for XIP) and provable SRAM bounds based on tensor lifetimes [19, 24].

1.8. Zant’s Positioning and Advantages

Zant is engineered to answer the shortcomings above with a narrow, embedded-first scope:

ONNX-native front end. Zant adopts ONNX as its native input format, directly parsing and compiling models exported from mainstream training frameworks such as PyTorch or TensorFlow. The Open Neural Network Exchange (ONNX) is an open standard designed to facilitate model interoperability across toolchains, defining a framework-agnostic representation of computational graphs through a standardized operator set and serialized protobuf-based format [25]. Its extensible design allows for consistent semantics between training and inference environments, even when the deployment backend is entirely decoupled from the training framework.

ONNX Runtime further formalizes support for post-training quantization workflows, including both the QDQ (Quantize–Dequantize) pattern and QOperator variants [21]. QDQ preserves explicit quantization nodes in the graph, enabling fine-grained precision control and compatibility with fake-quantization training flows. QOperator, on the other hand, replaces floating-point operators directly with quantized alternatives (e.g., `QLinearConv`), which are more suitable for hardware acceleration or low-level kernel mapping. Zant supports both paradigms and integrates seamlessly with calibration and export tools such as Microsoft Olive [20].

By consuming ONNX directly and preserving its semantics, Zant avoids the risk of semantic drift—discrepancies between training-time behavior and inference-time execution—which can otherwise emerge when converting models between loosely specified or custom formats. This design choice ensures that optimization passes, quantization strate-

gies, and operator behaviors remain consistent across the entire deployment pipeline.

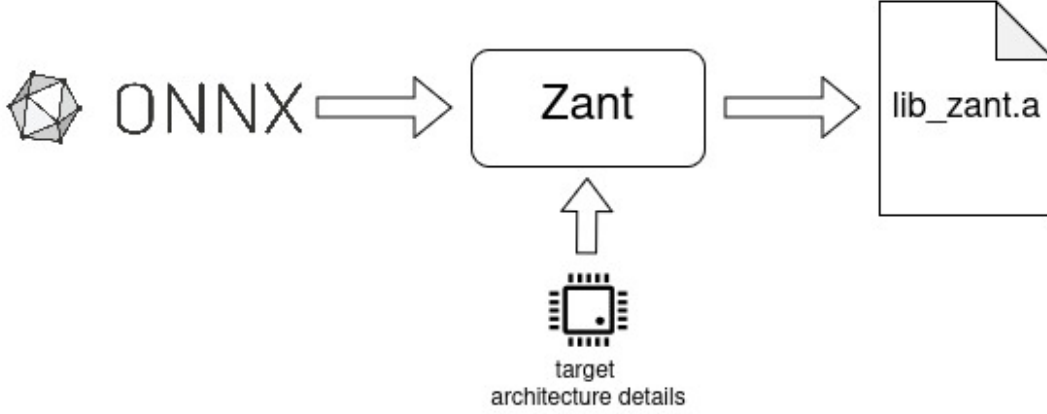


Figure 1.2: Detailed Zant workflow Schematic

Compile-time graph optimization and memory planning. Operator fusion (e.g., Conv–BN–ReLU) is applied before code generation time, eliminating intermediate materializations and reducing memory traffic in line with compiler literature [23]. Topological ordering and lifetime analysis yield a deterministic activation pool with a provable SRAM bound before flashing.

Section-aware linkage and XIP. Weights and constants are emitted to `.rodata` or flash-backed sections such as `.qspidata`, enabling execute-in-place on MCUs with QSPI/OSPI while reserving SRAM exclusively for activations [19, 24]. This directly addresses the dominant bottleneck identified in TinyML settings [17].

1.9. Quantization as a First-Class Optimization Axis

Quantization changes both the arithmetic and the memory/bandwidth profile of an inference workload. A standard linear quantizer maps real values r to integers via

$$Q(r) = \text{Int}\left(\frac{r}{S}\right) - Z, \quad \tilde{r} = S(Q(r) + Z), \quad (1.1)$$

where S is a positive scale and Z is the integer zero-point. Uniform quantization yields step size $S = (\beta - \alpha)/(2^b - 1)$ over clipping range $[\alpha, \beta]$ for b -bit precision [10, 22]. The choice between symmetric and asymmetric quantization reflects the statistical shape of the data (e.g., post-ReLU activations versus zero-centered weights), and the choice between per-tensor, per-axis, or blocked quantization reflects variation across channels or

sub-tensors.

From a memory perspective, the reduction scales ideally with precision: FP32→INT8 implies $\sim 4\times$ weight storage savings; INT4 can double that again [5]. From an energy perspective, integer MACs require less energy and have lower data-movement footprints [11]. Research on *mixed precision* and *outlier-aware* schemes further refines this trade-off, assigning higher precision to sensitive layers or patches while compressing tolerant regions [9, 27, 30].

In the ONNX ecosystem, both *QDQ* and *QOperator* are viable deployment targets. Zant adopts a *hybrid* stance: it accepts QDQ graphs for maximum compatibility and debuggability, then selectively replaces hot paths with operator-oriented implementations when specialized kernels provide large gains on the target MCU [21]. This policy preserves faithful semantics while allowing performance-oriented specialization where it matters most.

Hybrid quantization. Zant embraces QDQ for flexibility and debuggability, while supporting operator-oriented implementations where performance gains justify specialization on the target MCU [21]. The result is a practical reconciliation of accuracy retention and hardware efficiency [10, 22].

Portable performance via Zig with zero runtime dependencies. Zant’s artifacts are static, linkable libraries that integrate into C/C++ firmware without interpreters, allocators, or runtime parsers. Cross-compilation and conditional kernel selection provide portability across ARM Cortex-M, RISC-V, and x86 host harnesses used in CI [33?].

Developer ergonomics and verifiability. The CLI supports node extraction, test generation, and regression benchmarking so that correctness and performance regressions are caught early, and artifacts remain auditable down to function calls and linker sections [33].

1.10. ONNX as the Interoperability Backbone

The Open Neural Network Exchange (ONNX) has become the de facto standard for model interchange in state-of-the-art machine learning workflows. Rather than encoding framework-specific semantics, ONNX defines a portable operator set and a standardized serialization format that enables trained models to be exchanged across frameworks such as PyTorch, TensorFlow, and others [25]. This interoperability makes ONNX an ideal

foundation for deployment pipelines that must decouple training from inference, particularly in embedded contexts.

ONNX Runtime extends this foundation by introducing well-documented quantization pathways and execution providers, supporting two main representations for quantized models: the tensor-oriented *QDQ* (Quantize–Dequantize) pattern and the operator-oriented *QOperator* model [21]. The QDQ pattern, on the right of Figure 1.3, preserves explicit `QuantizeLinear` and `DequantizeLinear` nodes in the graph—faithful to fake-quantization workflows used during training—and allows fine-grained control over precision through per-tensor, per-axis, or blocked quantization. This explicit structure simplifies debugging and enables mixed-precision quantization schemes. In contrast, the QOperator, on the left of Figure 1.3, format replaces floating-point operators directly with quantized equivalents such as `QLinearConv` or `MatMulInteger`, which can be mapped more directly to optimized kernel libraries or hardware accelerators.

Together, these representations make ONNX a flexible and robust interoperability backbone for deploying models across a wide range of targets, from server-class hardware to highly constrained microcontrollers.

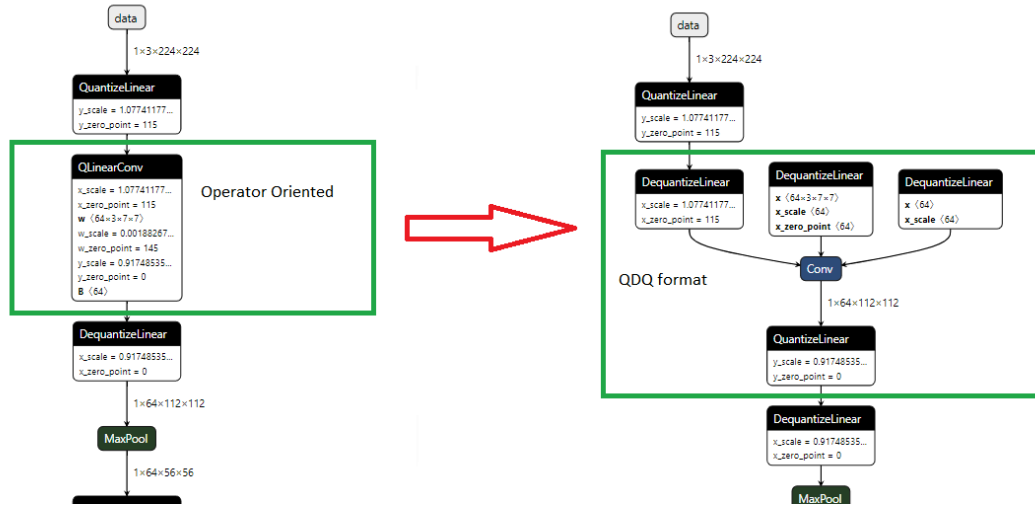


Figure 1.3: Two ONNX quantization representations. *Left:* operator-oriented (`QLinearConv`, `MatMulInteger`); *Right:* tensor-oriented QDQ with `QuantizeLinear`/`DequantizeLinear`. Zant adopts QDQ by default, with selective operator-oriented fast paths [21].

ONNX provides a unified, extensible graph representation designed to facilitate interoperability between training frameworks (like PyTorch or TensorFlow) and deployment backends, including custom runtimes. In Zant, ONNX serves not only as an interchange

format but as a contract: a well-defined, framework-agnostic specification from which optimized, platform-specific code can be generated deterministically. By building directly on the ONNX ecosystem, Zant preserves full interoperability with mainstream training frameworks while enabling embedded-specific optimizations such as operator fusion, lifetime-driven activation reuse, and memory section-aware placement.

1.11. Summary

Existing solutions have moved TinyML forward substantially: interpreters accelerate prototyping and broad hardware coverage; compilers demonstrate the upside of systematic optimization; vendor stacks and kernel libraries deliver critical per-operator speedups; and co-design validates memory-centric scheduling as a first-class technique. Zant builds on these insights but reshapes the deployment stack around the requirements that dominate MCU reality: static memory bounds, deterministic execution, flash-aware placement, and a quantization-first lowering from ONNX. In doing so, it offers a pragmatic, auditable path from trained models to firmware-ready artifacts, aligned with the workflows and constraints of embedded engineering.

2 | Methodology

The purpose of this chapter is to describe the methodology underlying Zant's toolchain and experimental setup. We detail the full compilation pipeline—from ONNX model ingestion to architecture-specific binary generation—emphasizing the design choices that enable deterministic, memory-safe deployment on microcontrollers. Each stage of the process, from model pre-processing and intermediate representation (IR) construction to optimization, memory planning, and code generation, contributes to ensuring that inference can be executed with statically proven resource bounds.

The chapter also explains how correctness and reproducibility are enforced: numerical equivalence against ONNX Runtime validates functional fidelity, while compile-time specialization and explicit sectioning guarantee predictability in both execution time and memory usage. Finally, we justify the adoption of Zig as the implementation substrate for the toolchain, outlining how its features align with the goals of determinism, portability, and tight control over system resources.

Notation. We use "peak SRAM" to mean the maximum simultaneously live activation memory at any point in the schedule. "Executable size" denotes the post-link flash footprint of the static library plus model-specific sections unless otherwise noted. "XIP" refers to execute-in-place usage of external flash through QSPI/OSPI memory mapping.

2.1. Zant Workflow

Before entering the details of each transformation stage, it is useful to outline the overall structure of the Zant workflow, which is summarized in Figure 2.1. The pipeline takes as input a neural network encoded in the ONNX format, treated as a hardware-agnostic description of the model's computational graph. Zant progressively transforms this representation into an optimized, target-specific inference library through a sequence of compilation phases, each building on the previous one.

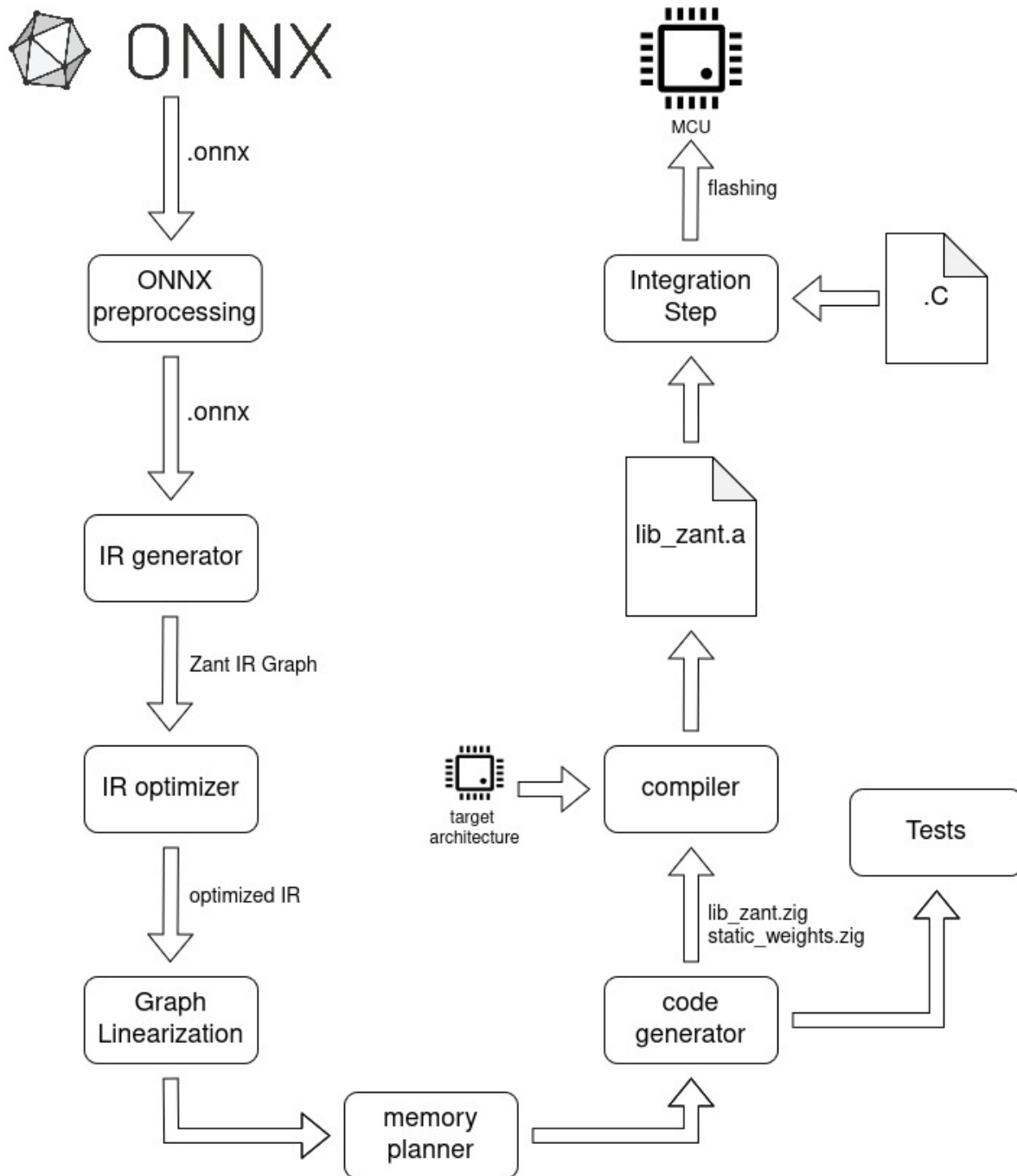


Figure 2.1: Zant toolchain schema

The workflow begins with the *model pre-processing* phase (Section 2.1.1), in which the raw ONNX file is simplified, cleaned of redundant or ambiguous structures, optionally quantized, and enriched with explicit shape information using ONNX Runtime’s inference engine. The resulting graph, now normalized and fully shape-resolved, is then translated into Zant’s *Intermediate Representation* (Section 2.1.2), a deployment-oriented abstrac-

tion where operators, tensors, and their attributes are made explicit to facilitate optimization.

Once the IR is constructed, an *optimization pass* (Section 2.1.3) applies pattern-matching techniques to fuse common operator sequences, eliminate redundant quantization–dequantization boundaries, and canonicalize layouts, producing a streamlined version of the graph. This optimized IR is subsequently *linearized* (Section 2.1.4) through a depth-first traversal that establishes a topologically valid execution sequence and reduces the lifetime of intermediate tensors.

With the execution order fixed, Zant performs *memory planning* (Section 2.1.5), assigning activation buffers either through a static lifetime analysis—yielding deterministic SRAM usage, or via a dynamic allocation strategy. The linearized and memory-planned graph is then lowered during the *code-generation phase* (Section 2.1.6) into architecture-neutral Zig source files that encode the kernel invocation sequence and declare all model parameters in the appropriate memory sections.

Finally, the generated sources are *compiled for the target architecture* (Section 2.1.8) using Zig’s cross-compilation facilities. This step emits a fully optimized static library tailored to the selected MCU or SoC, ready to be linked into embedded firmware. The following subsections describe each of these stages in detail.

2.1.1. Model pre-processing

Before any parsing or optimization, the input `.onnx` model undergoes a pre-processing stage to guarantee structural clarity, consistency of shapes, and readiness for deployment-oriented transformations. This phase involves three main steps: model simplification, optional quantization, and explicit shape materialization.

Model Simplification. The raw ONNX model, as exported from training frameworks such as PyTorch or TensorFlow, often contains redundant or opaque subgraphs introduced by frontend converters. These can include constant nodes for scalar parameters, duplicated reshapes, or fused operations decomposed into multiple primitives. To normalize the model, we employ `onnxsim` - a lightweight simplification tool built atop ONNX Runtime. The simplifier performs:

- Constant folding: evaluates static subgraphs at preprocessing time, embedding the results directly into the graph.
- Dead node elimination: removes tensors and operators not contributing to any

output.

- Shape propagation and inference: resolves ambiguous or symbolic dimensions when possible, ensuring each tensor has a well-defined static shape.
- Operator canonicalization: rewrites equivalent operator patterns into simpler or standardized forms. For example by replacing `Add(x, 0)` with `Identity(x)`.

This produces a minimal, deterministic graph structure that is easier to parse, optimize, and verify in later stages.

Quantization (Optional). When targeting microcontroller-class devices, we optionally quantize the simplified model to reduce memory footprint and computational cost. Zant supports *operator-oriented quantization*, in which ad-hoc operators allows 8 bit input tensors. Quantization can be performed using standard toolchains such as onnxruntime builtin quantization, configured for symmetric or asymmetric schemes and per-tensor or per-axis granularity.

Shape Materialization. Once simplified (and quantized, if applicable), the model is passed through a shape materialization phase. Here, every intermediate tensor between nodes is annotated with explicit shape metadata by invoking ONNX Runtime’s shape inference engine. This guarantees that all operator inputs and outputs have concrete, statically known dimensions at compile time. Zant’s downstream pipeline—particularly memory planning (Section 2.1.5) and code generation (Section 2.1.6)—relies on this information to allocate deterministic activation pools, align buffers, and emit fixed-size array declarations in generated Zig code.

The output of the pre-processing stage is therefore a simplified, optionally quantized, and fully shape-resolved ONNX graph that serves as the canonical input to Zant’s IR parser (Section 2.1.2).

2.1.2. IR Generator

In this step the ONNX graph is parsed into Zant’s deployment-oriented *Intermediate Representation (IR)*, a graph-based representation where the layers of the neural network are symbolized by the nodes of the graph, while the intermediate tensors between layers are represented by the edges of the graph. ONNX is treated as a descriptive format; **zant** IR is the substrate for optimization, scheduling, and code generation. The IR records:

- the network architecture: defining how nodes are connected.

- the node attributes: reference to input and output tensors, the mathematical operation they represent.
- the tensors and their attributes: shape, values if the tensor is a static attribute, it's category between input, intermediate, output or parameter, and quantization fields when present, like scale(s), zero-point(s), bit-width, per-tensor/per-axis as described in Section 1.9.

On an abstract level, the IR graph is represented as a directed graph $G = (V, E)$, where:

- $V = \{n_1, n_2, \dots, n_K\}$ is the set of nodes, each corresponding to one layer of the neural network;
- $E \subseteq V \times V$ is the set of directed edges, where each edge $(n_i \rightarrow n_j)$ indicates that the output tensor t_i of node n_i is used as an input tensor by node n_j .

In the Zant codebase, the IR graph is implemented as a set of objects, called **ZantNode**, each one representing a layer of the neural network.

```
IR = {ZantNode_1, ZantNode_2 ...}
```

Each **ZantNode** contains two lists of pointers to tensors, one for the input tensors and one for the output tensors, and another structure containing all the information that the mathematical operation needs.

```
ZantNode = {
    *ZantTensor[] input
    *ZantTensor[] output
    variables
}
```

Consequently, in the codebase, the edges of the graph are represented by the pointers to the tensors inside each node. A **ZantTensor** contains an array to represent the shape of the tensor and an array containing the values. From now on, we will discuss different operations on the IR. All of them will refer to **ZantNode**'s eliminations, repositioning, or to the insertion of a new **ZantNode** in the graph. So the graph's nodes are implicitly connected by their input and output tensors.

It is important to specify that only acyclic graph (DAG) structures are supported. The IR graph obtained after the `.onnx` parsing is not aware of any cyclic path, so before proceeding with the graph fusion and linearization, we check that the graph is a DAG (Direct Acyclic Graph) by navigating the structure and ensuring that it does not exist any node output that is the input of an already visited node.

Algorithm 2.1 Operator Fusion Example: Convolution $\rightarrow$ ReLU

Given: Input tensor x , weights W , bias b

Before Fusion:

$$y_1 = \text{Conv}(x, W) + b$$

$$y_2 = \text{ReLU}(y_1)$$

After Fusion:

$$y = \text{ReLU}(\text{Conv}(x, W) + b)$$

Effect: Combines two kernels into one; reduces memory traffic and launch overhead.

2.1.3. IR Optimizer

Starting from the Intermediate Representation obtained in the previous step, a *pattern-matching* algorithm pass identifies and fuses common operator sequences or subgraphs (e.g., the Convolution and ReLU blocks can be combined in a single Conv+ReLU block, as reported in Algorithm 2.1), removes redundant Q/DQ boundaries in operator-oriented graphs when safe, and canonicalizes layouts to the ABI (Application Binary Interface) expected by Zant’s kernels. Constant folding eliminates unnecessary materializations, reducing both flash and SRAM pressure.

The pattern matcher can operate 3 types of modification to the IR: the *fusion* of more nodes into a custom node, like in the case of Conv-BN-ReLU nodes fused into the single node Conv_BN_ReLU, to the *substitution* of a pattern with a well known node, like Dequant-Conv-Quant [1, 2, 4] into qLinearConv[3], or to the *elimination* of complementary node sequences like Dequant-Quant or Quant-Dequant [2, 4]. A schema of the three types of modification is present in Figure 2.2. What we obtain after this step is a reduced version of the IR that from now on we will call **opt_IR** (optimized Intermediate representation). If no fusion pattern has been detected, the **opt_IR** is identical to the IR.

2.1.4. Graph Linearization

Zant computes the nodes’ execution order using a depth-first search (DFS) traversal on the of the **opt_IR** graph. The purpose of this step is to produce a sequence of operator nodes that respects data dependencies while minimizing the live ranges of intermediate activations. When multiple traversal choices are possible, Zant applies a tie-break that prioritizes nodes closer to their consumers, thereby improving locality and reducing peak SRAM usage.

Formally, let the computation graph be represented as a DAG $G = (V, E)$, where:

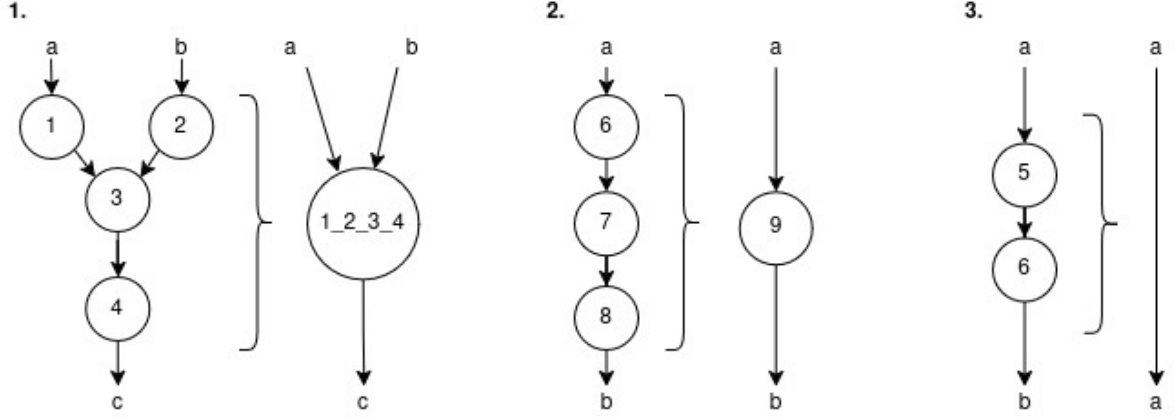


Figure 2.2: The three types of graph optimizations in Zant: **1. node fusion:** the pattern matcher detect a subgraph and fuse it into a custom made node; **2. node substitution:** the pattern matcher detect a subgraph and substitute it with an known node that does the equivalent operations; **3. node elimination:** when there is the mathematical proof that the input and output tensor of a sequence are identical the node sequence is deleted.

- $V = \{n_1, n_2, \dots, n_K\}$ is the set of **ZantNodes**, each corresponding to one mathematical operation;
- $E \subseteq V \times V$ is the set of directed edges, where each edge $(n_i \rightarrow n_j)$ indicates that the output tensor of node n_i is used as an input tensor by node n_j .

The linearization process produces a *topologically valid* sequence

$$\text{lin_IR} = \langle n_1, n_2, \dots, n_K \rangle \quad \text{such that} \quad \forall (n_i \rightarrow n_j) \in E : i < j,$$

meaning that each node appears only after all of its dependencies.

The obtained structure is an ordered list of **ZantNodes** where the **ZantNodes** properties are unchanged, see Figure 2.3 on the right. DFS algorithm 2.2 is chosen for its simplicity and its ability to preserve spatial and temporal locality in activation buffers.

Algorithm 2.2 Graph Linearization via Depth-First Search (DFS)

Input: DAG $G = (V, E)$

Output: topologically valid order lin_IR

initialize empty list lin_IR

initialize visited set $\mathcal{V} = \emptyset$

DFS(n)

if $n \in \mathcal{V}$ **then return**

 add n to $\mathcal{V}$

 let $\text{succ}(n)$ be outgoing neighbors of n

for $m \in \text{succ}(n)$ **do**

DFS(m)

 prepend n to lin_IR

for each source node s in G (no incoming edges)

DFS(s)

Result: $\text{lin_IR} = \langle n_1, n_2, \dots, n_K \rangle$ is a topologically valid order:

$$\forall (n_i \rightarrow n_j) \in E : \quad i < j$$

=0

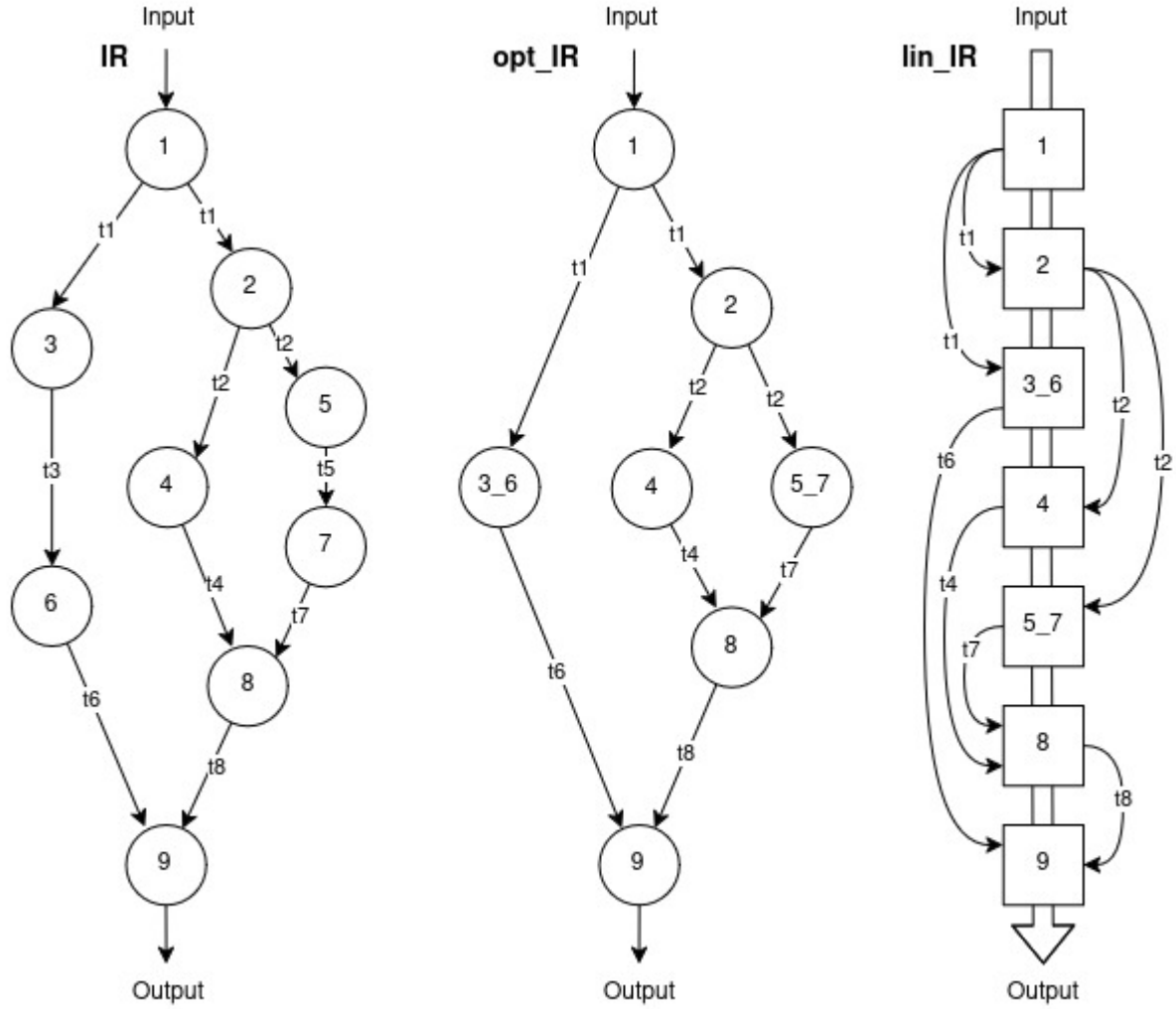


Figure 2.3: An example of the optimization process of the Zant Intermediate representations. On the left the *IR*, in the center the *opt_IR*, and on the right the *lin_IR*.

2.1.5. Memory Planning Strategies

Given the *lin_IR*, which contains the exact shapes of each intermediate tensor between nodes, Zant allow to choose between two memory allocation strategies:

- **Static allocation**, which performs compile-time lifetime analysis and buffer reuse.
- **Dynamic allocation**, which allocates and de-allocates tensors on the fly at runtime without prior memory planning.

This section focuses on the static strategy, which ensures deterministic SRAM usage and allows compile-time emission of memory bounds.

For each intermediate tensor t , we define:

- $b(t)$: the *birth index*, i.e., the index of the node that produces t .
- $d(t)$: the *death index*, i.e., the index of the last node that consumes t .
- $|t|$: the total number of elements in t .
- $type_size(t)$: the size in bytes of the element type of t (e.g., 1 for `int8`, 4 for `float32`).
- $size(t) = |t| \cdot type_size(t)$: the total size in bytes of t .

A *first-fit with reuse* heuristic 2.3 is used to place tensors in a shared memory pool, reusing space when lifetimes do not overlap. The algorithm maintains a list of free regions and greedily assigns new tensors to the first region that fits.

Algorithm 2.3 Static Pool Allocation (Greedy Reuse)

Tensors $T = \{t_1, t_2, \dots, t_n\}$ sorted by decreasing $size(t)$

Initialize empty **free_list**: a set of tuples (offset, size, free_from)

Initialize **pool_size** $\leftarrow 0$

For each tensor t in T :

Reclaim all regions in **free_list** where $free_from < b(t)$

If there exists a free region of size $\geq size(t)$:

Assign t to the first region large enough (aligned to A)

Else :

Place t at current **pool_size** (aligned); increase **pool_size** by $size(t)$

Add t 's region to **free_list** with $free_from = d(t)$

Return: Final memory layout; peak pool size $P^* = \text{pool_size}$

The output is a memory layout where each tensor is mapped to a non-overlapping region within a fixed-size activation pool. The peak SRAM required, P^* , is a compile-time constant reported into the generated code. This enables tight memory budgeting and deterministic inference behavior on microcontrollers.

In contrast, Zant's dynamic allocator (not detailed here) allocates and deallocates tensors at runtime. While this may reduce average memory usage in some cases, it introduces runtime overhead and non-determinism, which are undesirable in safety-critical or real-time embedded systems.

2.1.6. Code Generator

After Graph linearization and memory allocation, the optimized and linearized graph that can be seen as a sequence of mathematical operations where each operation takes as input one or more tensors and outputs another tensor, is lowered into two Zig source files: `lib_zant.zig`, which defines the `predict()` function and materializes the math kernel call sequence with explicit input and output buffer bindings for each node, and `static_parameters.zig`, which contains all parameter declarations annotated with section attributes for flash or execute-in-place (XIP) placement. These sources are then compiled into a static library (`.a`) that can be linked directly into embedded firmware.

All parameters—including weights, biases, and quantization scales or zero-points—are declared in read-only sections such as `.rodata` or `.qspidata`, which are mapped to flash memory, independently from the memory strategy used for intermediate tensors since they are static parameters that do not change between different inference sessions. Intermediate tensors, when going with the static memory strategy, are allocated in a dedicated SRAM section, typically in the `.bss` region, while the generated code itself resides in the `.text` section containing the kernel call sequence. A depiction of a standard organization of MCU memory segments can be found in Figure 2.4. Linker scripts define how these sections are mapped to the device’s physical memory regions. On XIP-capable platforms, the `.qspidata` mapping allows parameters to be fetched directly from flash through cache line fills, minimizing boot-time copies and conserving SRAM.

Beyond buffer reuse, the generated code explicitly leverages the memory hierarchy of the target MCU by implementing precise control over memory section usage. Weights and other read-only parameters are placed in sections such as `.rodata` or, when specified, in dedicated flash-backed segments like `.qspidata`, enabling execute-in-place (XIP) or read-in-place access through Quad Serial Peripheral Interface (QSPI) or Opta Serial Peripheral Interface (OSPI) interfaces instead of copying data into SRAM at startup [19, 24]. This approach allows the MCU to fetch parameters directly from flash, often with cache line fills that hide part of the access latency, freeing scarce SRAM for activations and temporary buffers while preserving deterministic access patterns. Activation pools are allocated in internal SRAM, and generated code is emitted into the `.text` section.

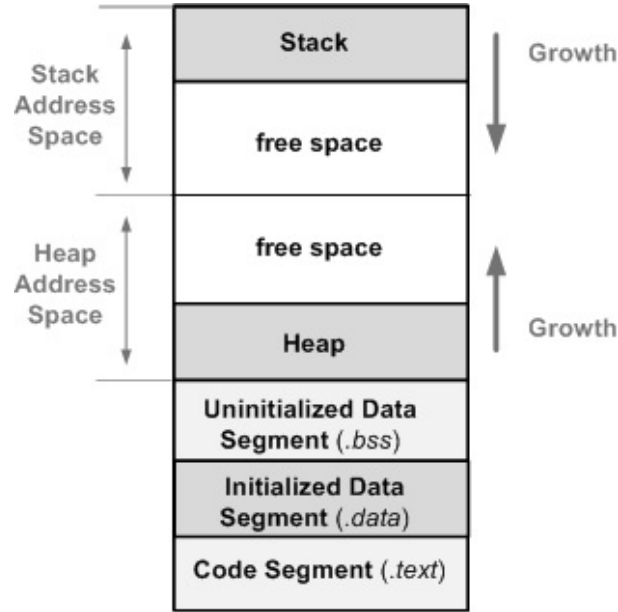


Figure 2.4: Memory sections and their layout. [26]

2.1.7. Correctness Validation Against ONNX Runtime

For each model and platform build, we validate numerical equivalence against ONNX Runtime (ORT) by feeding identical inputs and comparing outputs within type-appropriate tolerances (exact match for INT8; small margin for FP32). Any discrepancy triggers a debugging pass (e.g., per-node dumps) until alignment is achieved. Only models that pass this check are admitted to benchmarking.

2.1.8. Compiling for a Target Architecture

The last step of the Zant toolchain consists in taking the code-generated sources and compiling them for the chosen `target` and `cpu` using Zig (see Section 2.3). This phase transforms the architecture-neutral Zig code emitted during code generation into a fully optimized static binary tailored to the target microcontroller or SoC.

Zig’s cross-compilation capabilities allow Zant to produce architecture-specific artifacts without external toolchain dependencies. By passing the parameters `-Dtarget` and `-Dcpu` when calling compilation step we define both the instruction set and the microarchitectural features to be exploited. This ensures that the generated inference code is compiled with the same level of hardware awareness as hand-tuned embedded kernels. Zig natively supports a huge number of target architectures and CPUs, they can be seen by launching `zig targets` in your terminal.

During this phase, the linker integrates all memory sections defined in the code generation stage—`.text` for executable code, `.rodata` or `.qspidata` for static parameters to device-specific memory regions according to how you set them in your linker script. The compiler and linker toolchain map these sections to device-specific memory banks, allowing Zant-generated binaries to stream weights directly from flash on XIP-capable platforms with predictable latency, or to tune pool sizes and section placement at build time when XIP is unavailable or partially supported.

By leveraging Zig’s deterministic build system and fine-grained control over compilation flags, Zant guarantees reproducible builds and predictable binary layouts. The same model can thus be compiled across multiple targets—ARM Cortex-M, RISC-V, or custom DSP-based MCUs—while maintaining identical numerical behavior and ONNX-consistent semantics. This approach unifies code generation and deployment, producing self-contained inference libraries that integrate seamlessly into embedded toolchains.

The resulting static library (`libzant.a`) is therefore ready to be linked into a firmware project alongside the application logic.

2.2. Compile-Time Graphs Navigation for Deterministic Firmware

The compiler-first perspective brings two decisive advantages to MCU deployments. First, compile-time knowledge of graph topology, tensor shapes, and quantization parameters enables *lifetime analysis*: every intermediate tensor has a definable birth (production) and death (last consumption). A topological traversal yields a linear schedule and permits *activation reuse*, i.e., recycling buffer segments once their contents become dead. This converts a potentially unbounded dynamic footprint into a *statically proven* SRAM bound known before flashing firmware. Second, with code generation, the inference path becomes a sequence of *direct* function calls—no interpreter, no dynamic dispatch—reducing control overhead and jitter [17, 23].

Zant’s code generation pipeline reflects these priorities. The ONNX graph is parsed into an IR with explicit node/tensor metadata. Pattern-based fusion eliminates common sequences like `Conv`→`BatchNorm`→`ReLU` by reparameterizing weights and biases, removing intermediate materializations. The linearized schedule drives buffer allocation from fixed pools; allocation sizes are derived from shape metadata and alignment constraints known at compile time. Finally, Zant emits Zig code for the target architecture and links it into a static library for use in C/C++ projects [33].

The net effect is that *every byte* of SRAM consumed by inference is accounted for before deployment. This determinism extends naturally to execution time, improving schedulability in systems that multiplex inference with sensing and control tasks.

2.3. Zig as the compiler of choice

In the context of this methodology, we state what we use as the implementation substrate for Zant’s artifacts and why it satisfies the constraints of our evaluation. All results in this chapter are produced with a Zig-based code generator and runtime stubs that export a stable C ABI and static libraries consumable by MCU firmware.

Design Criteria Fixed for This Study

We require a language/tooling stack that provides: (i) deterministic, heap-free execution paths; (ii) compile-time specialization to model shapes and quantization metadata; (iii) explicit control of memory layout, alignment, and section placement for XIP; (iv) freestanding cross-compilation to Cortex-M; and (v) first-class C interoperability for integration into existing build systems.

Zig Features Used

- **Comptime specialization.** We use Zig’s *comptime* to specialize call sequences and small loops to concrete shapes/quantization parameters and to erase dead branches at build time.
- **Explicit allocation model.** All allocations are explicit (or omitted); activation buffers are emitted as static pools with compile-time known size, avoiding a heap in the inference path.
- **Precise layout and sectioning.** We rely on type/field alignment controls and link-section annotations to place parameters in flash-backed, read-only sections and activations in an SRAM-scoped section, with code in `.text` (XIP-friendly layout).
- **Freestanding cross-compiles.** We build for `thumb-freestanding` with CPU features pinned (e.g., `cortex_m7`) under a single `build.zig`, ensuring reproducible artifacts across host/MCU.
- **C ABI export/import.** We export a minimal C interface (`predict()`, `init/fini` where present) and can optionally import C kernels, allowing linkage into C/C++ firmware without adopting Zig project-wide.

- **Zero required runtime.** No exceptions/GC/RTTI; the resulting static libraries contain only the straight-line kernel calls and data needed by the evaluated models.

Impact on Methodological Ingredients

- **Artifacts under test.** The units we benchmark are Zig-produced static libraries (.a) plus headers. These are the measured objects for *executable size*, and the source of the *predict()* entry point timed on MCU.
- **Memory accounting.** The peak activation pool size is a compile-time constant emitted into the binary and linker map, which we report as *Peak SRAM*. Section annotations enable separate accounting of code, rodata (flash), and RAM.
- **Determinism controls.** The generated *direct-call* sequences eliminate interpreter dispatch and hidden allocations, aligning with our latency/jitter measurements.
- **Reproducibility.** A single `build.zig` controls targets/flags; we pin the Zig compiler version and record it in the reproducibility artifacts (Section ??).

Contextual Comparison

We note the alternatives considered for implementation of the artifacts under test: C (simple and universal but lacking integrated compile-time specialization), C++ (powerful templates but heavier build/runtime surfaces commonly disabled on MCUs), and Rust (strong safety, different build/FFI and codegen trade-offs). In this study we *use Zig* to meet the fixed criteria above while still exposing a C ABI to the rest of the firmware stack. We acknowledge a smaller ecosystem and evolving tooling relative to C/C++/Rust. To mitigate risk within this methodology, we (i) pin the Zig version, (ii) avoid third-party Zig dependencies in the hot path, and (iii) treat the C ABI and static libraries as the integration surface for all experiments. In summary, Zig is *the* implementation vehicle used to produce the static, section-aware, shape-specialized artifacts we benchmark. Its compile-time features, explicit memory model, freestanding cross-compilation, and C ABI support directly satisfy the constraints defined for our measurements without introducing a runtime that could confound determinism or memory accounting.

3 | Experimental Results

This chapter evaluates Zant across a representative suite of neural networks on two platforms, x86_64 and ARM Cortex-M7, focusing on inference time, peak memory usage, and executable size. Zig offers two optimized compiling strategies depending on the scope: `ReleaseFast` and `ReleaseSmall`. `ReleaseFast` enables aggressive compiler optimizations (like loop unrolling and vectorization) to make the code run as fast as possible, potentially increasing binary size. `ReleaseSmall` instructs the compiler to generate the smallest possible machine code, often sacrificing some speed to fit into constrained storage (critical for small microcontrollers). Here we compare Zant’s `static` and `dynamic` memory modes with `ReleaseSmall` optimization profile, in order to allow the smallest executable possible, and contrast Zant against the interpreter-based baseline *TensorFlow Lite Micro* (TFLM) [7].

The model suite is grouped into the categories used in this chapter:

- **Core models:** `mnist-8`, `mobilenet_v2`, `beer`, `coco80_q`
- **DSCNN variants:** different types of models, all obtained by a sequence of operators blocks designed ad hoc for testing purposes.

Model Descriptions

mnist-8 A compact fully connected network trained on the MNIST handwritten-digit dataset. It operates on grayscale inputs of size $1 \times 1 \times 28 \times 28$ and produces a 10-dimensional class probability vector. Its simplicity and small memory footprint make it a standard baseline for evaluating inference engines on constrained devices.

mobilenet_v2 A convolutional architecture built from depthwise-separable convolutions and inverted residual blocks. In our evaluation, it takes RGB inputs of size $1 \times 3 \times 32 \times 32$ and outputs 4 class logits. Thanks to its favourable accuracy-efficiency trade-off, MobileNetV2 represents realistic embedded vision workloads.

beer A lightweight object-detection model that predicts bounding boxes and class scores on a fixed 12×12 grid. The network follows a compact convolutional design and is representative of MCU-friendly detection architectures in which both spatial resolution and model complexity are constrained for real-time operation.

DSCNN models The family of DSCNN models used in this evaluation is generated through a parameterized TensorFlow/Keras script that constructs depthwise-separable convolutional networks of varying width and depth. Each model consists of an initial 3×3 convolutional stem followed by a sequence of depthwise-pointwise blocks. Each block applies a depthwise convolution, Gemm, ReLU activation, and a 1×1 pointwise convolution followed by a second normalization and activation, following a *MobileNet-style* design, an example is reported in Figure 3.1. Two scalar parameters govern the architecture: a width multiplier α , which scales the number of output channels in all convolutional layers, and a block count N , which determines the network depth. By sweeping α and N over a predefined set of configurations, we obtain a smooth progression of DSCNN models with increasing parameter count, activation footprint, and computational cost. For every configuration, the script exports both a TensorFlow `SavedModel` (used for ONNX conversion and numerical validation) and a FP32 `.tflite` model (used to assess TFLM deployability). Since we are working on both x86_64 and Cortex-m7 we created two different models batch: one with smaller models for the cortex-m7 (Table 3.2) and one for x86 (Table 3.1). This yields to a controlled and diverse model suite for stress-testing memory usage, inference latency, and binary size across toolchains.

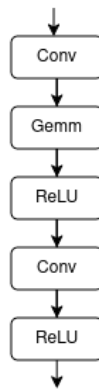


Figure 3.1: Conceptual schema of the block repeated in DSCNN models.

All models were validated for numerical correctness using ONNX Runtime prior to benchmarking.

Table 3.1: DSCNN Model Configurations

Model Name	Number of Blocks	Width Multiplier α	dimension (KB)
dscnn_2	2	0.1	3.93
dscnn_4	4	0.2	9.03
dscnn_6	6	0.4	132.75
dscnn_8	8	0.8	1814.76
dscnn_10	10	1.0	4907.13
dscnn_12	12	1.2	10013.11
dscnn_14	14	1.4	17635.16
dscnn_16	16	1.6	28322.46
dscnn_18	18	1.8	42456.96

The tag `_deep` represents models that have a larger number of layers.

Table 3.2: tiny DSCNN Model Configurations

Model Name	Number of Blocks	Width Multiplier α	dimension (KB)
tiny_S	5	0.2	10.02
tiny_M	5	0.3	17.33
tiny_L	5	0.7	42.3
tiny_S_deep	6	0.5	30.01
tiny_M_deep	8	0.5	55.38
tiny_L_deep	12	0.5	80.35

3.1. Zant Static vs Dynamic Memory Allocation: Inference Time on x86_64

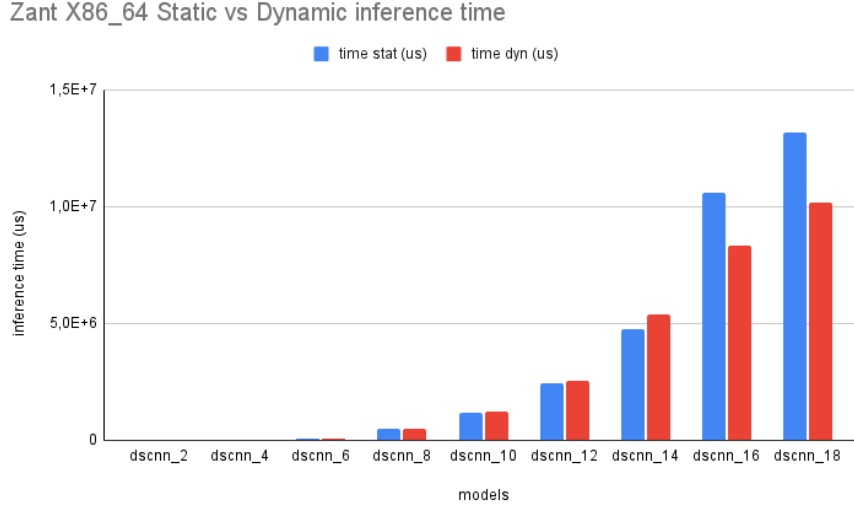


Figure 3.2: Comparison of inference time across models using static and dynamic memory allocation strategies.

Figure 3.2 shows the inference time for each model on a X86_64 architecture. As we can better see in Figure 3.3 dynamic allocation outperforms static allocation for the two smallest and two largest models. On average, dynamic memory allocation on X86_64 architecture yields a reduction in inference time, outperforming static allocation by a factor of approximately $1.157\times$.

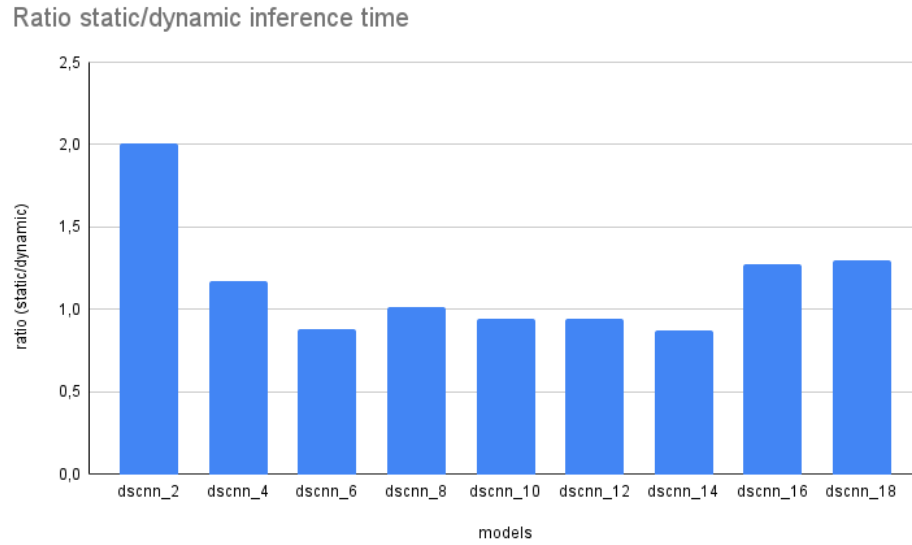


Figure 3.3: Ratio of static to dynamic inference time for each model.

3.2. Zant Static vs Dynamic Memory Usage: RAM x86_64

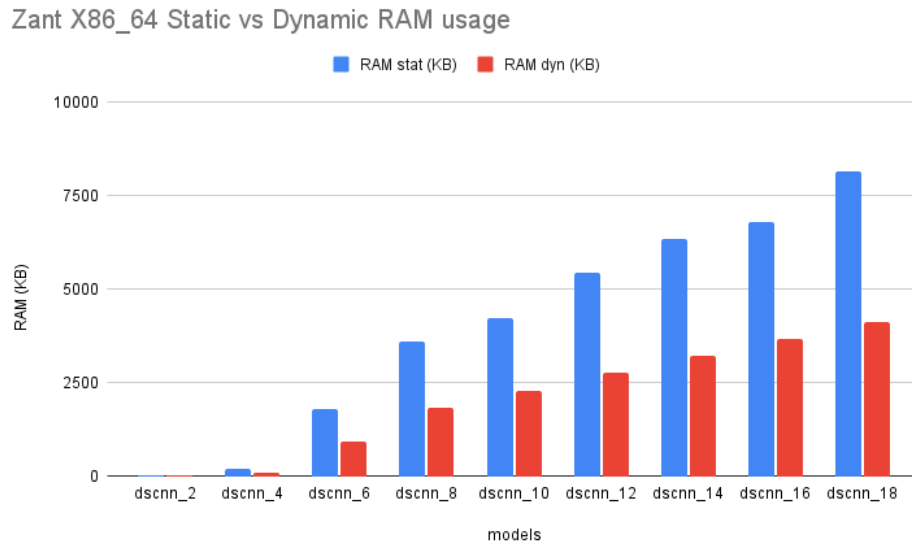


Figure 3.4: Peak RAM usage across models under static and dynamic memory allocation.

On x86_64, memory usage differs significantly between allocation modes. Figure 3.4 illustrates that:

- **Dynamic mode:** Allocates tensors on demand, minimizing memory footprint at the cost of runtime planning.
- **Static mode:** Pre-allocates reusable buffers, leading to higher peak memory usage due to alignment padding and buffer lifetime overestimation, as explained in Section 2.1.5.

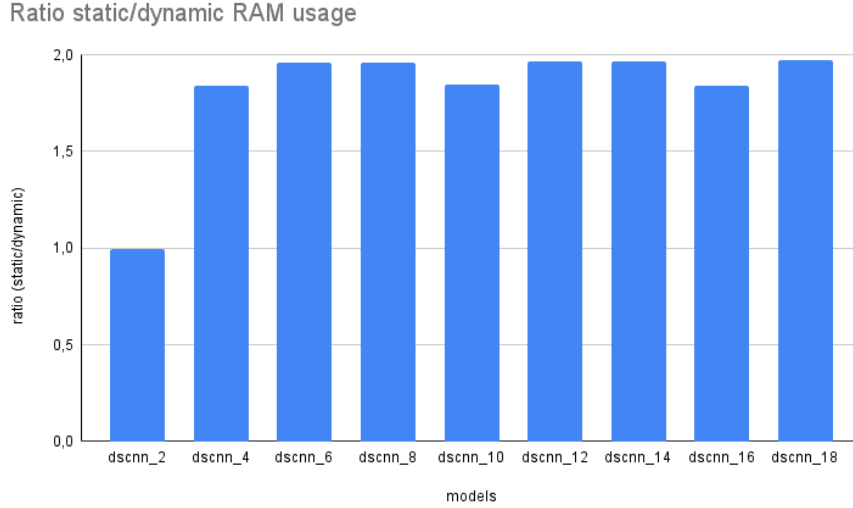


Figure 3.5: Ratio of static to dynamic RAM usage for each model.

As shown in Figure 3.5, the static mode uses significantly more memory. On average, Zant’s dynamic allocator achieves a $1.815\times$ lower peak RAM usage compared to static allocation, demonstrating its efficiency on host platforms. However static allocation ensures deterministic SRAM usage and allows compile-time emission of memory bounds. For a detailed explanation on how memory allocation strategies works see Section 2.1.5.

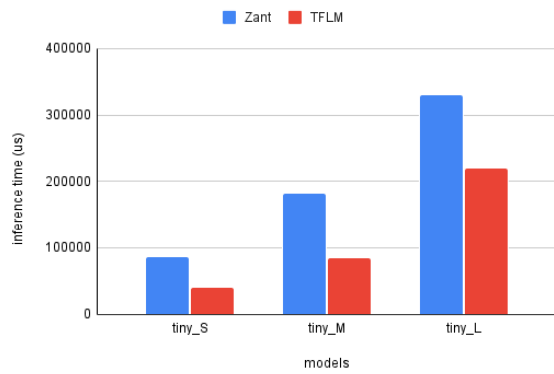
3.3. Zant vs TensorFlow Lite Micro (TFLM) on ARM Cortex-M7

This section compares Zant against the interpreter-based baseline *TensorFlow Lite Micro* (TFLM) across three deployment-critical metrics: inference time, peak RAM consumption, and executable size. The evaluation focuses on two families of convolutional models that were specifically designed to fit within the memory constraints of ARM Cortex-M7 targets without relying on execute-in-place (XIP). The first family, denoted `tiny`, increases the number of channels per layer while keeping network depth fixed; the second, `tiny_deep`, follows the opposite strategy and increases the number of layers while pre-

serving channel width. These complementary configurations allow us to examine how different architectural patterns, wide versus deep models, interact with each inference engine and whether Zant and TFLM exhibit distinct behaviours on a microcontroller-class device. For Zant deployment we used the dynamic memory allocation strategy 2.1 since the static strategy is still immature to cover all the benchmarking models with such limited hardware constraints.

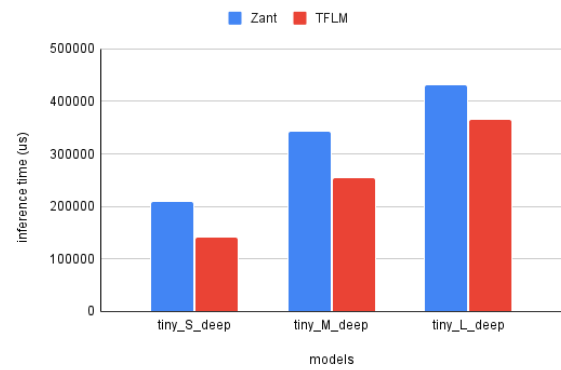
Inference Time

Zant Dynamic vs TFLM inference time
Cortex-m7



Tiny models time comparison.

Zant Dynamic vs TFLM inference time
Cortex-m7



Deep tiny models time comparison.

Figure 3.6: Comparison of inference time between Zant (dynamic) and TFLM on ARM Cortex-M7.

Zant Dynamic vs TFLM inference time

Cortex-m7

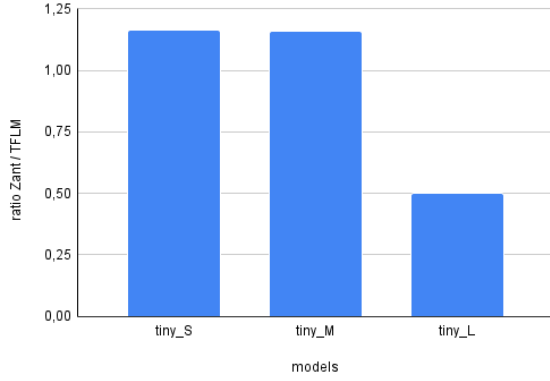


Figure 3.7: Tiny models ratio.

Zant Dynamic vs TFLM inference time

Cortex-m7

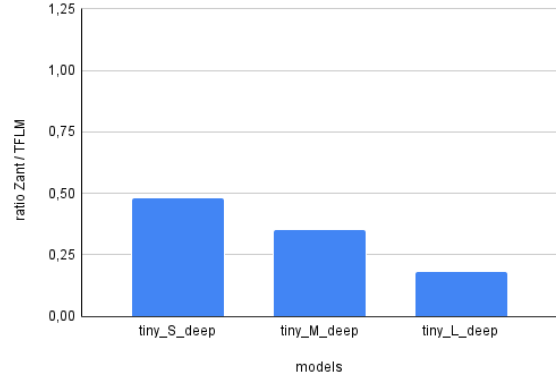


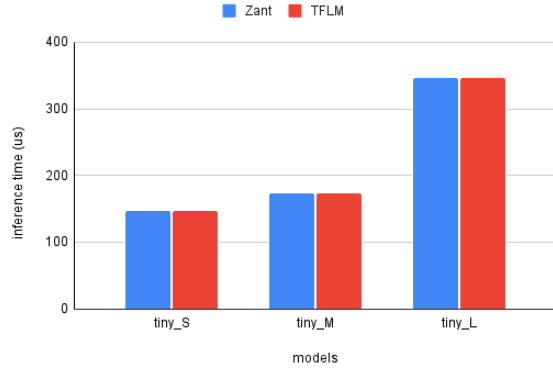
Figure 3.8: Deep tiny models ratio.

Figure 3.9: Ratio between Zant and TFLM inference time for tiny models. The ratio is computed in the following way $Ratio = \frac{Zant_time}{tflm_time} - 1$

We measure the execution times of Zant and TFLM over the **dscnn** class of models, and report the results in Figure 3.6, and the ratio between the execution times of the two frameworks on the same network in Figure 3.9. As shown in Figures 3.6 and 3.9, TFLM achieves consistently faster inference times compared to Zant. On average, TFLM is approximately $1.638\times$ faster on Cortex-M7, likely due to its kernel-level optimizations and handcrafted operator implementations. We can also notice that the difference is more relevant for the first category of models. This means that TFLM has more optimized kernels for wide convolutions with respect to the one of Zant. Furthermore, there is a descending trend in Figure 3.9, which means that the deeper the network, the less relevant the TFLM kernel optimization becomes. At the same time it is due to a larger number of fused kernels on the Zant side that for deeper networks have a bigger impact.

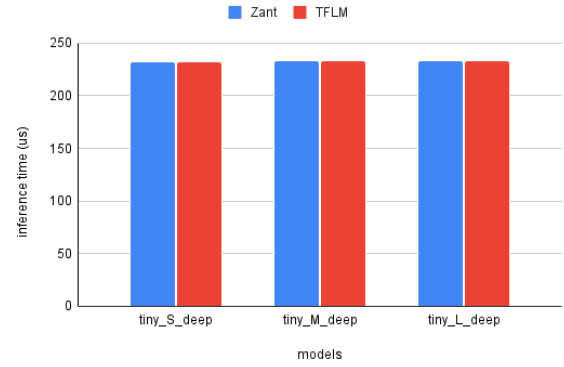
Peak RAM memory Usage

Zant Dynamic vs TFLM RAM usage
Cortex-m7



Tiny models peak RAM comparison.

Zant Dynamic vs TFLM RAM usage
Cortex-m7



Deep tiny models peak RAM comparison.

Figure 3.10: Comparison of peak RAM memory usage between Zant (dynamic) and TFLM on ARM Cortex-M7.

Zant Dynamic vs TFLM RAM usage
Cortex-m7

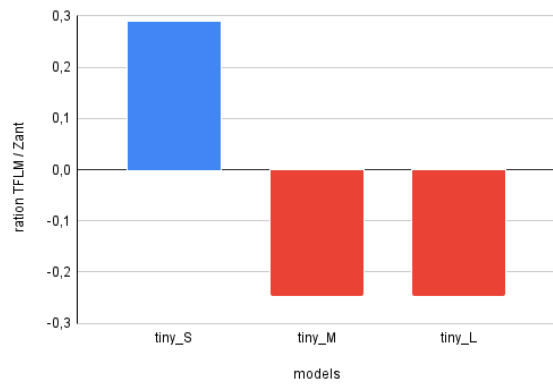


Figure 3.11: Tiny models ratio.

Zant Dynamic vs TFLM RAM usage
Cortex-m7

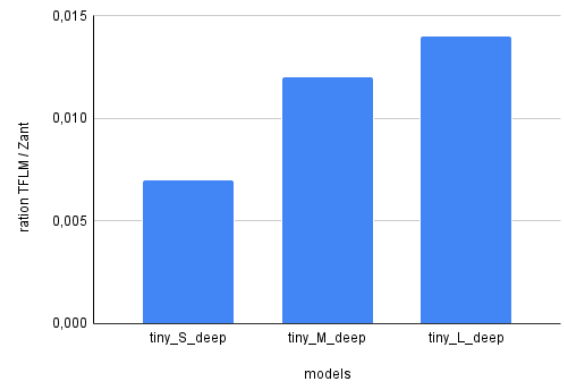


Figure 3.12: Deep tiny models ratio.

Figure 3.13: Ratio between Zant and TFLM peak RAM memory usage for tiny models. The ratio is computed in the following way $Ratio = \frac{Zant_peakRAM}{tflm_peakRAM} - 1$

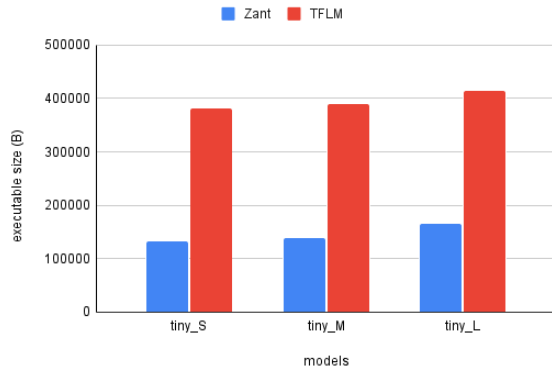
Despite its higher inference latency, Zant maintains competitive memory usage on Cortex-M7. As shown in Figure 3.10, the average RAM consumption of Zant on a cortex M7 is comparable to TFLM. The average ratio of TFLM over Zant is 0.9715, indicating that

TFLM has a marginally lower peak memory usage, but the difference is minor. For `_deep` models the peak RAM usage is almost the same for all the models since the max width of the model is the same across all of them.

Executable Size

Zant Dynamic vs TFLM executable size

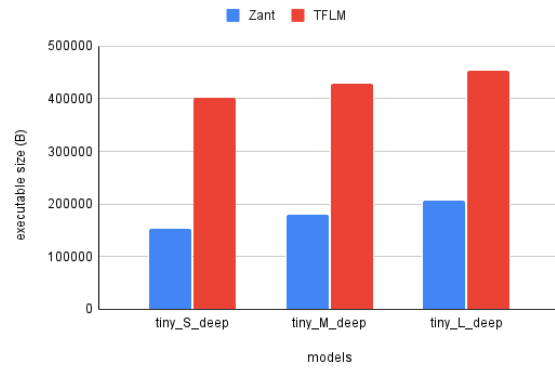
Cortex-m7



Tiny models time comparison.

Zant Dynamic vs TFLM executable size

Cortex-m7



Deep tiny models time comparison.

Figure 3.14: Comparison of executable size between Zant (dynamic) and TFLM on ARM Cortex-M7.

Zant Dynamic vs TFLM executable size

Cortex-m7

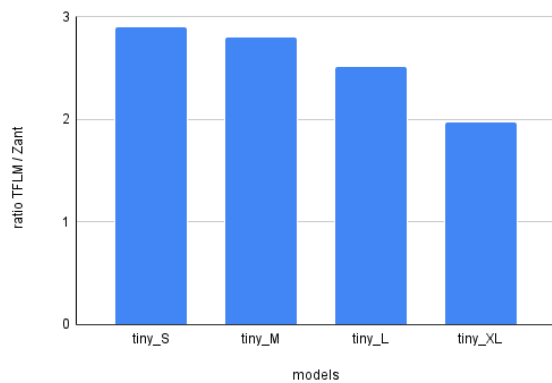


Figure 3.15: Tiny models ratio.

Zant Dynamic vs TFLM executable size

Cortex-m7

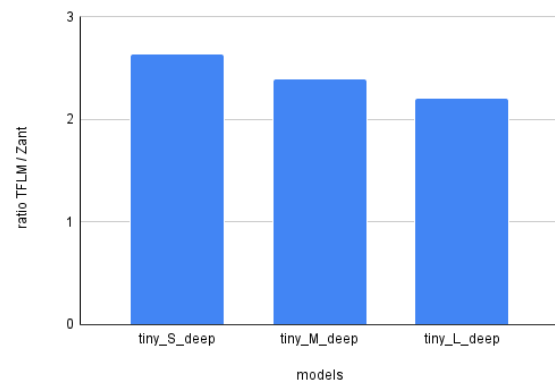


Figure 3.16: Deep tiny models ratio.

Figure 3.17: Ratio between Zant and TFLM executable size for tiny models. The ratio is computed in the following way $Ratio = \frac{Zant_exeSize}{tflm_exeSize} - 1$

The above charts reports a comparison for each model between the executable size obtained with Zant and the one obtained with TFLM. As illustrated in Figure 3.14 Zant demonstrates a significant advantage in binary size. Zant produces smaller executables than TFLM across all tested models. On average, the Zant-generated binaries are $2.432\times$ smaller, making Zant especially suitable for flash-constrained embedded systems. We can notice how the ratio (Figure 3.17) decreases with the growth of the model size, which is likely due to the fact that the code for runtime interpretation of the graph has a bigger influence for small models in TFLM.

Real-world models on MCU

Tables 3.3 and 3.5 report the performance of additional representative models across two architectures (x86_64 and Cortex-M7) and two memory allocation modes (Static and Dynamic). These models span a wide range of computational profiles—from small fully-connected networks (mnist-8 [8]), to mid-size convolutional architectures (mobilenet_v2 [16]), to application-specific networks with FOMO backbone (beer [12]), and finally larger quantized object-detection backbones (coco80_q).

Table 3.3: x86_64 Performance (Static vs Dynamic Memory)

Model	Zant Static		Zant Dynamic		Tensorflow	
	Peak Mem	Time (μ s)	Peak Mem	Time (μ s)	Peak Mem	Time (μ s)
mnist-8	30.625 KB	1721	49.59 KB	3581	2688 KB	983
mobilenet_v2	256 KB	12371	120.1 KB	15454	fail	fail
beer	369 KB	15781	553.7 KB	28032	fail	fail
coco80_q	248.125 KB	310495	113.7 KB	461227	fail	fail

Table 3.4: ARM Cortex-m7 TFLM vs Zant

Model	Zant		TFLM	
	pass	XIP used	pass	notes
mnist-8	True	False	True	
mobilenet_v2	True	True	False	model overflow flash mem
beer	True	False	False	Missing Operator
coco80_q	True	True	False	Missing Operator

Table 3.5: Cortex-M7 Performance (Static vs Dynamic Memory). **.a Size** in the Byte size of the static library; **#IS** is the number of instructions necessary to run the program once, **.bin Size** is the size of the binary, when a "+" is present it means that we have two .bin files, one for the main flash memory and one for the XIP on the external flash memory; **Time (μs)** is the average time needed for inference.

Model	Static				Dynamic			
	.a Size	#IS	.bin Size(B)	Time (μs)	.a Size	#IS	.bin Size(B)	Time (μs)
mnist-8	110K	61580	151K	81862	124K	63253	155K	83273
mobilenet_v2	7,0M	56164	6,9M + 144K	1689926	7,0M	57189	6,9M + 147K	1679494
beer	101K	65359	163K	709873	92 KB	66570	167K	709006
coco80_q	9,2M	55995	9,1M + 141K	12218000	9,2M	56178	9,1M + 141K	12405977

The results illustrate several important trends:

Execution time benefits from static memory

While static mode reduces memory control overhead the speedups are model-dependent. For lightweight models (mnist-8), static inference is roughly $2\times$ faster on x86_64, whereas for larger convolutional networks the gap narrows. On Cortex-M7, execution times are nearly identical between modes because both ultimately perform the same sequence of kernels; however, static mode still avoids many runtime allocation.

Large models highlight the role of XIP (Execute-In-Place)

For models such as `mobilenet_v2` and `coco80_q`, Zant leverages QSPI/OSPI XIP to access weights directly from external flash without loading them into SRAM. This is critical for M7-class devices, where internal SRAM is limited (typically $< 1\text{--}2$ MB). XIP enables these models to run at all, whereas fully SRAM-resident layouts would be impossible.

Binary size and instruction count

On Cortex-M7, the static and dynamic variants produce very similar binary sizes and instruction counts, since both ultimately compile the same kernel calls. Minor differences arise from the presence or absence of dynamic-allocator scaffolding. In large models, the binary size is dominated by the weight blobs embedded in `.rodata` unless XIP is used.

TFLM baseline limitations

As summarized in Table 3.4, two of the evaluated models (`beer`, `coco80_q`) do not execute on TFLM due to missing operators. This reflects a structural constraint of TFLM: only a subset of TensorFlow Lite’s kernels is available in the *micro* runtime, and applications must explicitly register each needed op via the `MicroMutableOpResolver`. In practice the “ground truth” list of what you can use is whatever appears in `micro_ops.h` [32] and

`micro_mutable_op_resolver.h` [31] ; many advanced or rarely used ops are absent, so models that rely on them will not load without implementing and linking custom kernels.

3.4. Discussion of Experimental Results

The experimental results provide quantitative evidence for the design choices made in Zant.

3.4.1. Static vs Dynamic Allocation in Zant

On x86_64, dynamic allocation reduces peak activation memory compared to the static planner. In terms of latency, static allocation is never worse and often faster on x86_64. The absence of runtime allocation logic and the improved locality from a stable layout help reduce control-plane overhead. The dynamic mode remains useful as a development tool (e.g., when rapidly iterating on operators or shapes), but the results of real-world models clearly support static planning as the preferred choice for deployed MCU firmware.

On the Cortex-M7 platform, inference times between static and dynamic variants are very close, since both ultimately dispatch the same kernel sequence. However, static mode still provides the crucial advantage of a proven compile-time SRAM bound and simplified reasoning about worst-case memory usage. This is especially relevant for systems where inference shares SRAM with sensing, control, and communication tasks.

3.4.2. Zant vs TensorFlow Lite Micro

The comparison with TFLM highlights complementary strengths:

- **Inference time.** TFLM achieves lower latency on Cortex-M7 for the subset of models it supports, typically outperforming Zant by a factor of around $1.5\times$ – $1.7\times$. This advantage stems primarily from highly optimized kernels and years of engineering effort focused on ARM microcontrollers.
- **Peak SRAM.** Zant’s peak SRAM usage is comparable to that of TFLM. The differences between the two are small on the evaluated models, indicating that both systems employ reasonably effective activation planning. Zant’s distinguishing factor is that its SRAM bound is derived analytically at compile time rather than configured via a manually tuned arena size.
- **Executable size.** Zant produces substantially smaller binaries than TFLM. On Cortex-M7, Zant’s executables are on average more than twice as small as their

TFLM counterparts. For flash-constrained deployment scenarios, this reduction can be decisive, especially when the same firmware must host multiple models or additional application logic.

- **Model coverage.** Some models in the evaluation suite (`beer` and `coco80_q`) cannot be deployed with TFLM due to missing operator support. Zant, by generating code from ONNX graphs, is not constrained by a fixed built-in operator set: new operators can be added incrementally as IR patterns and kernel implementations. This flexibility is particularly important for real-world applications that rely on bespoke architectures rather than canonical benchmarks.

Operator coverage interacts tightly with quantization. TFLM’s most mature and efficient path targets 8-bit quantized models (INT8 activations and weights), often with per-channel weight quantization for (depthwise) convolutions; those kernels are prioritized because they map well to MCU SIMD libraries such as CMSIS-NN.

Integration effort is also non-trivial. Beyond converting the model, developers must (i) register every operator used (or fall back to the broader—and heavier—`AllOpsResolver`), (ii) provision and tune a static tensor arena, and (iii) implement or port any missing ops as custom operators with the required TFLM registration boilerplate. This increases engineering time and can bloat the binary when many ops are pulled in, which is why it is common to see TFLM projects fail fast at runtime when an op is missing, as observed in our baseline.

Taken together, these results show that Zant does not yet match the raw kernel-level performance of a mature interpreter like TFLM, but it offers strong advantages in binary size, determinism, extensibility, and model coverage. For many MCU deployments, especially those with tight flash budgets or non-standard models, these properties can be more impactful than absolute latency.

3.4.3. Scalability Across Model Families

The DSCNN model families demonstrate that Zant’s approach scales smoothly with both depth and width. As the number of blocks and channels grows, the static planner continues to produce feasible activation pools, and the XIP-aware sectioning keeps parameter storage within flash limits. The experiments confirm that activation memory, rather than parameter size, is the primary bottleneck on MCUs, and that compile-time scheduling and fusion can significantly mitigate this constraint.

4 | Conclusions and future developments

This thesis has addressed the problem of deploying deep neural networks on resource-constrained microcontrollers by proposing *Zant*, an ONNX-native, Zig-based inference engine that compiles models into deterministic, memory-efficient static libraries. The work argues that MCU-scale inference is best treated as a compile-time problem where graph semantics, memory planning, and code generation are handled jointly, rather than as a runtime concern delegated to an interpreter.

By designing and implementing *Zant*, and by evaluating it on representative models and platforms, the thesis demonstrates that it is possible to (i) obtain precise static bounds on SRAM usage, (ii) exploit execute-in-place (XIP) to keep large models feasible, and (iii) remain interoperable with the mainstream ONNX ecosystem, all while producing artifacts that integrate cleanly into embedded firmware workflows.

4.1. Summary of Contributions

The main contributions of this thesis can be summarized as follows:

- **ONNX-native compile-time deployment pipeline.** *Zant* parses ONNX graphs into a deployment-oriented intermediate representation, applies pattern-based optimizations (fusion, substitution, elimination), and emits architecture-specific Zig code. The toolchain preserves ONNX semantics, supports both QDQ and QOperator quantization schemes, and interoperates with existing calibration and export tools.
- **Lifetime-driven static memory planning.** The thesis formalizes a static memory allocation strategy based on tensor lifetime analysis over a linearized DAG schedule. Intermediate activations are placed into a single shared pool with buffer reuse, yielding a compile-time peak-SRAM bound that is exported into the generated code. This turns activation memory from a heuristic design concern into a

verifiable property.

- **Section-aware memory layout and XIP support.** Zant introduces explicit memory sectioning for weights, activations, and code: parameters are emitted into read-only sections (e.g., `.rodata`, `.qspidata`) suitable for execute-in-place, activations are placed into SRAM-only pools, and code into `.text`. This layout allows models with large parameter sets to stream weights from external flash on XIP-capable MCUs, freeing SRAM for activations.
- **Deterministic, auditable inference libraries.** Instead of relying on an interpreter, Zant generates straight-line, direct-call inference code that can be compiled to static libraries with a stable C ABI. This design minimizes control-plane overhead, avoids hidden heap usage, and yields artifacts that are straightforward to inspect, profile, and integrate with embedded CI pipelines.
- **Empirical evaluation across models and platforms.** The thesis evaluates Zant on a suite of models, including fully connected networks (mnist-8), convolutional architectures (mobilenet_v2), application-specific models (beer), and larger quantized backbones (coco80_q), as well as parametric DSCNN families. Experiments span x86_64 and a Cortex-M7 MCU, and compare Zant’s static and dynamic allocation modes against TensorFlow Lite Micro (TFLM) in terms of inference time, peak SRAM, and executable size.

4.2. Hardware Awareness Without Lock-In

Zant leverages hardware features across architectures without vendor lock-in. On ARM Cortex-M it can exploit DSP extensions and CMSIS-NN kernels; on RISC-V or heterogeneous SoCs it supports equivalent SIMD or accelerator libraries through compile-time configuration. This modular design positions Zant between interpreter-based systems like TFLM and compiler-based stacks like TVM: ONNX-native, deterministic, and selectively optimized.

While designed for single-device inference, Zant also integrates naturally into distributed setups such as split computing and early exiting, ensuring compact and deterministic local execution. ONNX compatibility simplifies partitioning and independent compilation of subgraphs across devices.

4.3. Distributed and Collaborative Inference: Context for MCUs

Although Zant targets single-device inference, edge deployments often participate in larger systems. *Split computing* and *early exiting* are two paradigms in which a portion of a model (or an early classifier) runs on-device, with the remainder offloaded to an edge server or cloud when inputs are hard or when policies dictate [18]. Surveys and empirical studies report significant latency and energy gains by co-designing partition points and early exits, particularly under variable network conditions. While many of these systems focus on mobile or gateway-class hardware, the conceptual lesson carries over: making the *easy* cases cheap on-device is a robust strategy.

Zant’s contribution here is foundational rather than prescriptive: by ensuring compact, deterministic on-device inference for the portion of the graph that *must* run locally, it complements split/exit strategies without imposing a particular partitioning method. Interoperability with ONNX further eases cross-device graph surgery, since partitioned subgraphs can be re-exported and compiled independently.

4.4. Limitations

The current Zant prototype has several limitations that frame the scope of the conclusions:

- **Operator coverage.** Only a subset of ONNX operators is supported, primarily those required by the evaluated models. Broader coverage (including recurrent layers, attention mechanisms, and more exotic activations) remains future work.
- **Kernel maturity.** Zant’s kernels are intentionally simple and portable. Apart from optional integration with libraries like CMSIS-NN, they do not yet exploit the full range of microarchitectural optimizations (e.g., hand-tuned assembly, advanced tiling, or aggressive vectorization). This explains much of the performance gap with TFLM.
- **Fixed-shape assumption.** The lifetime analysis and static planning assume fixed tensor shapes known at compile time. Models with dynamic shapes, variable-length sequences, or input-dependent control flow are outside the current design.
- **Hardware diversity.** Experiments are limited to x86_64 (as a proxy host) and a single Cortex-M7 platform. While the design is portable to RISC-V and other MCUs, the results should not be over-generalized without further validation.

Acknowledging these limitations is important: Zant should be interpreted as a *design point* that demonstrates the feasibility and benefits of compile-time planning, not as a fully productized replacement for existing frameworks.

4.5. Future Developments

The work opens several avenues for future research and engineering:

4.5.1. Kernel Optimization and Backend Integration

A natural next step is to integrate highly optimized kernel libraries more systematically. On ARM, deeper use of CMSIS-NN and other vendor-specific intrinsics could narrow the performance gap with TFLM. On RISC-V and other architectures, analogous libraries or custom SIMD kernels could be developed. In the longer term, Zant could select kernels per target via a pluggable backend interface, preserving portability while exploiting hardware features.

4.5.2. Richer Graph Transformations and Scheduling

The pattern-based optimizer can be extended with more aggressive transformations, such as:

- more complex fusion patterns and user-made fusion pattern,
- tiling and streaming strategies that explicitly trade recomputation for lower peak activations,
- alternative scheduling heuristics optimized for specific memory/cache hierarchies.

Combining such transformations with the existing lifetime analysis could further reduce SRAM usage, especially for deeper CNNs and attention-based models.

4.5.3. Dynamic-Shape Support Under Bounded Resources

Supporting limited forms of dynamic shapes (e.g., variable batch size or sequence length within a bounded range) is an interesting challenge. One possibility is to generate multiple static plans and select among them at runtime, or to compute conservative upper bounds on lifetimes and pool sizes based on symbolic shape analysis. This would extend Zant’s determinism to a wider class of models without reverting to fully dynamic allocation.

4.5.4. Energy-Aware Evaluation and Co-Design

Future experimental campaigns should include direct energy measurements using power analyzers, especially on battery-powered boards. Combining Zant’s compile-time planning with energy models could enable:

- automatic selection of quantization and layout strategies to minimize energy per inference,
- co-design of networks and deployment plans similar to MCUNet, but within an ONNX-centric toolchain.

4.5.5. Tooling, Diagnostics, and Developer Experience

Finally, there is significant room to improve Zant’s ergonomics:

- richer diagnostics (e.g., visualizations of lifetimes and buffer reuse),
- automated reporting of SRAM/flash budgets per model,
- integration with common build systems and IDEs,
- higher-level APIs that let developers express deployment constraints (e.g., “max 128 KiB SRAM”) and have Zant search for feasible quantization and scheduling configurations.

4.6. Closing Remarks

This thesis has shown that a compiler-first, ONNX-native approach to TinyML is not only feasible but advantageous in several dimensions that matter for microcontrollers: static memory guarantees, flash footprint, and auditability. Zant does not try to replace general-purpose frameworks; rather, it complements them by translating trained models into deterministic, deployment-ready artifacts that respect the realities of embedded systems.

By treating inference as a compile-time problem and using ONNX as the semantic backbone, Zant provides a foundation on which more sophisticated optimizations, kernels, and deployment strategies can be built. The hope is that this work contributes to making TinyML deployments more predictable, transparent, and accessible to embedded engineers who must balance machine learning ambition with tight, non-negotiable resource constraints.

Bibliography

- [1] . Onnx conv documentation, . URL https://onnx.ai/onnx/operators/onnx__Conv.html. Comprehensive guide to Conv.
- [2] . Onnx dequantizelinear documentation, . URL https://onnx.ai/onnx/operators/onnx__DequantizeLinear.html. Comprehensive guide to Dequantize-Linear.
- [3] . Onnx qlinearconv documentation, . URL https://onnx.ai/onnx/operators/onnx__QLinearConv.html. Comprehensive guide to QLinearConv.
- [4] . Onnx quantizelinear documentation, . URL https://onnx.ai/onnx/operators/onnx__QuantizeLinear.html. Comprehensive guide to QuantizeLinear.
- [5] C. Banbury, V. Janapa Reddi, M. Lam, W. Fu, A. Fazel, J. Holleman, X. Huang, R. Hurtado, D. Kanter, A. Lokhmotov, et al. Mlperf tiny benchmark. *arXiv preprint arXiv:2106.07597*, 2021. URL <https://arxiv.org/abs/2106.07597>.
- [6] T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, M. Cowan, H. Shen, L. Wang, Y. Hu, L. Ceze, C. Guestrin, and A. Krishnamurthy. Tvm: An automated end-to-end optimizing compiler for deep learning. In *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 578–594. USENIX, 2018. URL <https://www.usenix.org/system/files/osdi18-chen.pdf>.
- [7] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang, P. Warden, and R. Rhodes. Tensorflow lite micro: Embedded machine learning for tinymml systems. *Proceedings of Machine Learning and Systems*, 3:800–811, 2021.
- [8] H. S. E. M. M. Z. X. Deng. Mnist handwritten digit classification based on convolutional neural network with hyperparameter optimization. *researchgate*, 2023. URL https://www.researchgate.net/publication/369265604_MNIST_Handwritten_Digit_Classification_Based_on_Convolutional_Neural_Network_with_Hyperparameter_Optimization.

- [9] L. Deng, G. Li, S. Han, L. Shi, and Y. Xie. Model compression and hardware acceleration for neural networks: A comprehensive survey. *Proceedings of the IEEE*, 108(4):485–532, 2020.
- [10] A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney, and K. Keutzer. A survey of quantization methods for efficient neural network inference. *arXiv preprint arXiv:2103.13630*, 2022. URL <https://arxiv.org/abs/2103.13630>.
- [11] M. Horowitz. 1.1 computing’s energy problem (and what we can do about it). *IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*, pages 10–14, 2014.
- [12] E. impulse. Fomo, 2024. URL <https://docs.edgeimpulse.com/studio/projects/learning-blocks/blocks/object-detection/fomo>. Accessed Oct. 14, 2025.
- [13] K. Kääriäinen and Contributors. onnx2c: Onnx to c compiler for tinymml, 2025. URL <https://github.com/kraiskil/onnx2c>. Accessed Oct. 14, 2025.
- [14] Y. Kang, J. Hauswald, C. Gao, A. Rovinski, T. Mudge, J. Mars, and L. Tang. Neurosurgeon: Collaborative intelligence between the cloud and mobile edge. In *Proceedings of the 22nd International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 615–629. ACM, 2017. URL https://www.cl.cam.ac.uk/~ey204/teaching/ACS/R244_2023_2024/papers/kang_asplos_2017.pdf.
- [15] L. Lai, N. Suda, and V. Chandra. Cmsis-nn: Efficient neural network kernels for arm cortex-m cpus. *arXiv preprint arXiv:1801.06601*, 2018. URL <https://arxiv.org/abs/1801.06601>.
- [16] K. D. C. Z. Y. R. Y. Li. Mobilenetv2 model for image classification. *ieeexplore*, 2020. URL <https://ieeexplore.ieee.org/document/9422058>.
- [17] J. Lin, W.-M. Chen, Y. Lin, C. Gan, and S. Han. Mcunet: Tiny deep learning on iot devices. In *Advances in Neural Information Processing Systems*, volume 33, pages 11711–11722. Curran Associates, Inc., 2020.
- [18] Y. Matsubara, E. Osmankovic, M. Levorato, A. Mandalari, A. Mathur, R. Kolcun, and H. Haddadi. Split computing and early exiting for deep learning applications: Survey and research challenges. *ACM Computing Surveys*, 2022. URL <https://arxiv.org/abs/2103.04505>. Preprint: arXiv:2103.04505.
- [19] Microchip Technology. Execute in place (xip) with quad spi interface (qspi), 2016. URL <https://ww1.microchip.com/downloads/en/DeviceDoc/>

- Atmel-44065-Execute-in-Place-XIP-with-Quad-SPI-Interface-SAM-V7-SAM-E7-SAM-S7_Application-Note.pdf. Application Note; Accessed Oct. 14, 2025.
- [20] Microsoft. Onnx quantization passes in microsoft olive, 2024. URL https://microsoft.github.io/Olive/0.7.0/features/passes/quant_onnx.html. Accessed Oct. 14, 2025.
- [21] Microsoft. Onnx runtime quantization documentation, 2024. URL <https://onnxruntime.ai/docs/performance/model-optimizations/quantization.html>. Comprehensive guide to QDQ and QOperator formats.
- [22] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. Van Baalen, and T. Blankevoort. A white paper on neural network quantization. *arXiv preprint arXiv:2106.08295*, 2021. URL <https://arxiv.org/abs/2106.08295>.
- [23] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren. Dnnfusion: Accelerating deep neural networks execution with advanced operator fusion. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI)*, pages 883–898, New York, NY, USA, 2021. ACM. doi: 10.1145/3453483.3454083.
- [24] NXP Semiconductors. Smart vehicles need smart memory architectures: The need for execute-in-place (xip), 2021. URL <https://www.nxp.com/docs/en/white-paper/SMART-VEHICLES-XIP-WP.pdf>. White paper; Accessed Oct. 14, 2025.
- [25] ONNX Community. Open Neural Network Exchange (ONNX), 2025. URL <https://onnx.ai/>. Accessed Oct. 14, 2025.
- [26] Open4Tech. Memory layout of embedded c programs, 2025. URL <https://open4tech.com/memory-layout-embedded-c-programs/>. Website visited november 13th 2025.
- [27] M. Rusci, A. Capotondi, and L. Benini. Memory-driven mixed low precision quantization for enabling deep network inference on microcontrollers. *arXiv preprint arXiv:1905.13082*, 2024. URL <https://arxiv.org/html/1905.13082>.
- [28] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu. Edge computing: Vision and challenges. *IEEE Internet of Things Journal*, 3(5):637–646, 2016. doi: 10.1109/JIOT.2016.2579198.
- [29] G. Tagliavini, S. Mach, D. Rossi, C. Haugou, P. Flatresse, and L. Benini. A trans-precision floating-point platform for ultra-low power computing. *IEEE Transac-*

tions on Computers, 68(9):1377–1390, 2019. doi: 10.1109/TC.2018.2872062. URL <https://arxiv.org/abs/1711.10374>.

- [30] W. Tao, J. Wang, Z. An, and J. Xiao. Value-driven mixed-precision quantization for patch-based inference on microcontrollers. *arXiv preprint arXiv:2401.13714*, 2024. URL <https://arxiv.org/abs/2401.13714>.

- [31] tensorflow. tensorflow/lite/micro/micro_mutable_op_resolver.h, .URL.

tensorflow. tensorflow/lite/micro/kernels/micro_ops.h, .URL.

Zant Foundation. Z-ant: A comprehensive, open-source neural network framework, 2025. URL <https://github.com/ZantFoundation/Z-Ant>. Zant inference engine repository and documentation.

List of Figures

1.1	MCUNet co-design: TinyNAS (architecture search) + TinyEngine (memory-adaptive inference). Demonstrates joint hardware–software optimization for MCU deployment [17].	10
1.2	Detailed Zant workflow Schematic	12
1.3	Two ONNX quantization representations. <i>Left</i> : operator-oriented (<code>QLinearConv</code> , <code>MatMulInteger</code>); <i>Right</i> : tensor-oriented QDQ with <code>QuantizeLinear</code> / <code>DequantizeLinear</code> . Zant adopts QDQ by default, with selective operator-oriented fast paths [21].	14
2.1	Zant toolchain schema	18
2.2	The three types of graph optimizations in Zant: 1. node fusion : the pattern matcher detect a subgraph and fuse it into a custom made node; 2. node substitution : the pattern matcher detect a subgraph and substitute it with an known node that does the equivalent operations; 3. node elimination : when there is the mathematical proof that the input and output tensor of a sequence are identical the node sequence is deleted.	23
2.3	An example of the optimization process of the Zant Intermediate representations. On the left the <i>IR</i> , in the center the <i>opt_IR</i> , and on the right the <i>lin_IR</i>	25
2.4	Memory sections and their layout. [26]	28
3.1	Conceptual schema of the block repeated in DSCNN models.	34
3.2	Comparison of inference time across models using static and dynamic memory allocation strategies.	36
3.3	Ratio of static to dynamic inference time for each model.	37
3.4	Peak RAM usage across models under static and dynamic memory allocation.	37
3.5	Ratio of static to dynamic RAM usage for each model.	38
3.6	Comparison of inference time between Zant (dynamic) and TFLM on ARM Cortex-M7.	39
3.7	Tiny models ratio.	40
3.8	Deep tiny models ratio.	40

3.9	Ratio between Zant and TFLM inference time for tiny models. The ratio is computed in the following way $Ratio = \frac{Zant_time}{tflm_time} - 1$	40
3.10	Comparison of peak RAM memory usage between Zant (dynamic) and TFLM on ARM Cortex-M7.	41
3.11	Tiny models ratio.	41
3.12	Deep tiny models ratio.	41
3.13	Ratio between Zant and TFLM peak RAM memory usage for tiny models. The ratio is computed in the following way $Ratio = \frac{Zant_peakRAM}{tflm_peakRAM} - 1$	41
3.14	Comparison of executable size between Zant (dynamic) and TFLM on ARM Cortex-M7.	42
3.15	Tiny models ratio.	42
3.16	Deep tiny models ratio.	42
3.17	Ratio between Zant and TFLM executable size for tiny models. The ratio is computed in the following way $Ratio = \frac{Zant_exeSize}{tflm_exeSize} - 1$	42

List of Tables

3.1	DSCNN Model Configurations	35
3.2	tiny DSCNN Model Configurations	35
3.3	x86_64 Performance (Static vs Dynamic Memory)	43
3.4	ARM Cortex-m7 TFLM vs Zant	43
3.5	Cortex-M7 Performance (Static vs Dynamic Memory). .a Size in the Byte size of the static library; #IS is the number of instructions necessary to run the program once, .bin Size is the size of the binary, when a "+" is present it means that we have two .bin files, one for the main flash memory and one for the XIP on the external flash memory; Time (μs) is the average time needed for inference.	44

Acknowledgements

It all started as a joke a year and a half ago. A dear colleague of mine told me about a new programming language, Zig. He said we should build something together. After a bit of brainstorming, we decided to dive into tinyML. That's how Zant began. Over time, it has gone through many changes and is constantly evolving, but here are a few things it has taught me that will never change: the environment you find yourself in, and the way you live it, are fundamental to achieving your goals; the real resource is people, not lines of code; and last but not least, dream big. All of this was possible thanks to a special place, the Politecnico di Milano, and the people who walk its corridors, driven by curiosity and the desire to challenge themselves. I would like to thank my family for their support and closeness, and all my friends for their encouragement and the laughs. I would also like to thank Prof. Fabio Salice for his supervision and Dott. Massimo Pavan for his technical guidance and support throughout the development of this thesis.

(PS: Zant was originally going to be called Guernica, in honor of the BCL room where the first lines of code were written.)

